Design and Implementation of a 4-Bit RISC CPU Datapath with a 3-Stage Pipeline, Using Verilog HDL

Project Report: Theory, Implementation and Verification

Item	Details
Student / team	LE DINH SON
Guide	Shivashish Pal
Institution	FPT University
Duration	1 weeks
Tools	Xilinx Vivado 2026.1 (Vivado Simulator, XSim); Icarus Verilog 12
Target device	Artix-7 xc7a12ticsg325-1L
Date	October 2026

Abstract

This report describes the design, implementation and verification of a 4-bit RISC CPU written in Verilog HDL. The CPU has four 4-bit registers, 12-bit instructions, a 16 x 12-bit instruction memory, a 16 x 4-bit data memory, a hardwired control unit and a 3-stage pipeline (IF / ID / EX). Every block is a separate Verilog module that was tested on its own and then connected in `cpu_top`. Four self-checking testbenches ran 88 checks (11 ALU + 11 register file + 22 control unit + 44 CPU) and all passed, with no unknown (X) values on the key outputs after reset. Once the pipeline is full, one instruction finishes every clock cycle; a taken jump costs 2 extra cycles.

The report has two parts. **Part A** (chapters 1-8) covers theory and design. **Part B** (chapters 9-16) covers the implementation, verification, simulation results, the exact workflow followed in Vivado, the problems met on the way and how each was solved.

Part A. Theory and Design

1. Introduction

A Central Processing Unit (CPU) reads instructions from memory and carries them out one after another. Every CPU, from a small microcontroller to a desktop processor, is built from two halves:

- **The datapath** stores, selects and processes values: registers, the ALU, memories, multiplexers and the buses (groups of wires) that connect them.

- **The control path (control unit)** looks at each instruction and drives the select, enable and write-control signals of the datapath so that the datapath does the right thing.

"4-bit" means the registers, the ALU and the data memory all work with 4-bit numbers (0 to 15). The CPU uses a 3-stage pipeline: while one instruction is being executed, the next one is being decoded and the one after that is being fetched. The design is small enough to understand completely, yet it contains every important feature of a real processor: fetch, decode, execute, memory access, write-back, flags, jumps and pipelining.

1.1 Aim

To design and implement a functional 4-bit RISC CPU datapath using Verilog HDL and verify its operation through simulation and a structured testbench.

1.2 Objectives

1. Design a 4-bit arithmetic and logic unit (ALU) supporting arithmetic and bitwise operations.
2. Implement a four-register general-purpose register file.
3. Design the program counter, instruction memory, instruction register and datapath multiplexers.
4. Develop a hardwired control unit that decodes instructions and generates datapath control signals.
5. Organise the CPU as a 3-stage pipeline (fetch, decode, execute) and handle jumps correctly.
6. Integrate the modules into a top-level CPU and execute short programs.
7. Verify functional behaviour using directed test cases and simulation waveforms.

1.3 Key specifications

Parameter	Specification
Processor type	Simple 4-bit RISC-style CPU, load/store architecture
Datapath width	4 bits (registers, ALU operands, data memory)
Register file	4 general-purpose registers R0-R3, 4 bits each; 2-bit register numbers
Instruction width	12 bits: opcode, two register fields and a 4-bit immediate
Instruction memory	16 words x 12 bits
Program counter	4 bits; addresses 16 instruction words
Data memory	16 words x 4 bits (for LOAD / STORE)
Pipeline	3 stages: IF (fetch), ID (decode), EX (execute + memory + write-back)

Parameter	Specification
Speed	One instruction finishes every clock after the pipeline fills; a taken jump costs 2 extra clocks
Control	Hardwired: combinational decoder + branch unit
Flags	Carry (arithmetic) and Zero (result = 0000)
Implementation	Verilog HDL, RTL simulation (Vivado Simulator / Icarus Verilog)
Project duration	2 weeks

Table 1. Key specifications of the CPU

2. Computer Architecture Fundamentals

2.1 Instruction Set Architecture (ISA)

The ISA is the agreement between software and hardware. It says which instructions exist, how each one is written in binary, which registers a programmer can use and how memory is addressed. In this project the 12-bit instruction format and the opcode table were fixed first (chapter 6), and the hardware was then designed to carry them out. This is a teaching CPU, so it is not compatible with any commercial instruction set.

2.2 RISC versus CISC

Feature	RISC (Reduced Instruction Set)	CISC (Complex Instruction Set)
Instructions	Few, simple, regular	Many, some very complex
Instruction length	Fixed (here: always 12 bits)	Variable
Memory access	Only LOAD and STORE touch memory	Most instructions can use memory
Arithmetic operands	Registers (or a small immediate)	Registers or memory
Decoding hardware	Simple, fast, hardwired	Complex, often microprogrammed
Pipelining	Easy, because every instruction looks alike	Harder
Examples	ARM, RISC-V, MIPS	x86

Table 2. RISC versus CISC

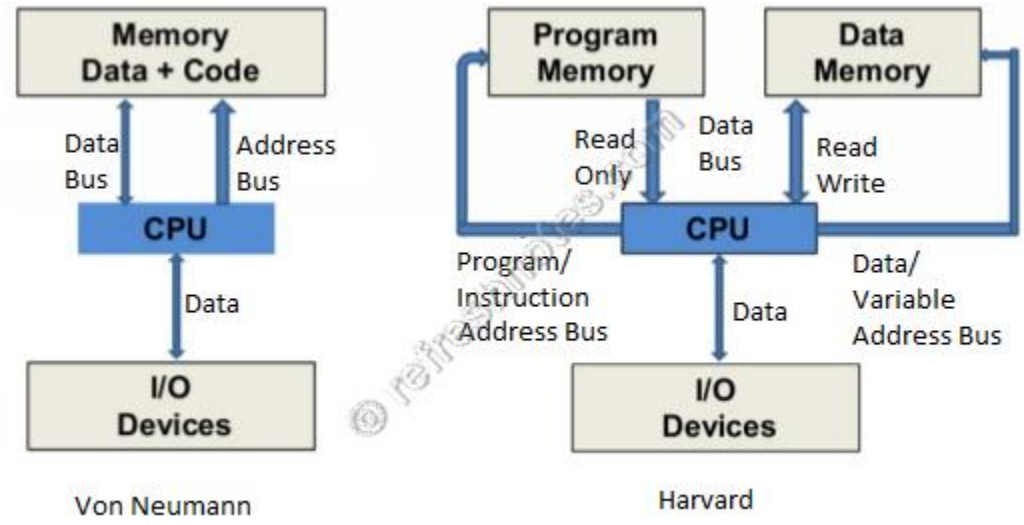
This CPU follows the RISC idea. The most important result is the load/store architecture: the ALU never works directly on memory. To add two numbers that are in memory, a program first loads them into registers, adds the registers, and then stores the answer back:

```
LOAD  R0, 3    ; R0 = data_memory[3]
LOAD  R1, 4    ; R1 = data_memory[4]
ADD   R0, R1   ; R0 = R0 + R1
STORE R0, 5    ; data_memory[5] = R0
```

2.3 Von Neumann versus Harvard memory

- **Von Neumann:** one memory holds both instructions and data on one shared bus. The CPU cannot fetch an instruction and read or write data at the same moment.
- **Harvard:** instructions and data are kept in two separate memories, each with its own bus. A data store can never overwrite the program, and the two memories can have different word sizes.

The Harvard model matters especially for a pipeline: in the same clock cycle the IF stage reads the instruction memory while the EX stage may be reading or writing the data memory. With one shared memory these two would clash. This CPU uses a 16 x 12-bit instruction memory and a separate 16 x 4-bit data memory.



3. The Instruction Cycle and Pipelining

3.1 The steps of every instruction

1. **Fetch:** the PC addresses instruction memory and the instruction is loaded into the IR.
2. **Decode:** the control unit reads the opcode and makes the control signals; the register and immediate fields are separated.
3. **Operand read:** the register file gives the values of the source registers.
4. **Execute:** the ALU does the arithmetic or logic operation.
5. **Memory access:** LOAD or STORE uses the data memory.
6. **Write-back:** the result is written into the destination register.
7. **Next instruction:** the PC moves to PC + 1, or to a jump address.

In this CPU the seven steps are grouped into three pipeline stages: **IF** = step 1, **ID** = step 2 and **EX** = steps 3 to 7.

3.2 Three ways to use the clock

Design style	How it works	Clocks for 4 instructions	Comment
Single-cycle	All steps in one long clock cycle	4 (but each clock is long)	No separate fetch register
Multi-cycle (no pipeline)	One step-group per clock, one instruction at a time	12 (3 stages x 4)	Hardware is idle most of the time
3-stage pipeline (this CPU)	One stage per clock; all stages busy with different instructions	6	One instruction finishes every clock once the pipeline is full

Table 3. Comparison of CPU timing organisations

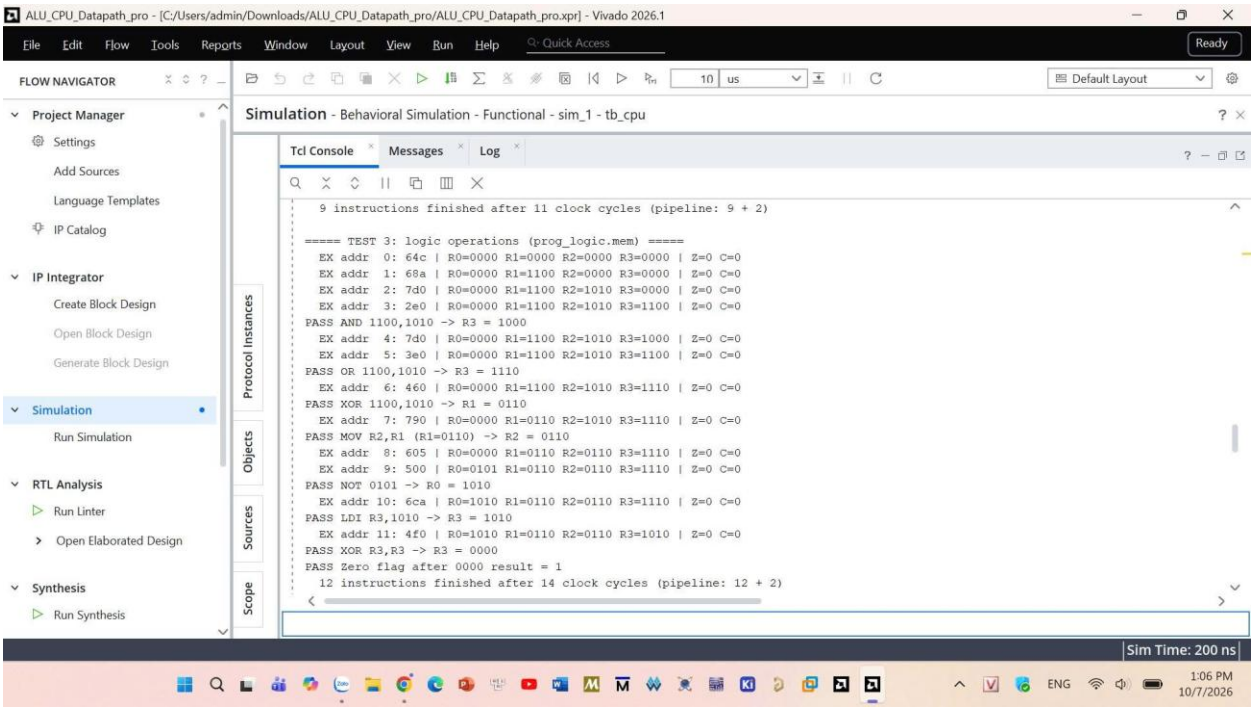
3.3 What is a pipeline?

A pipeline works like an assembly line. Instead of one worker building a whole product before starting the next, the job is split into stations; each station does its part and passes the product on, and all stations work at the same time on different products. In each clock cycle of this CPU:

- IF fetches instruction n + 2,
- ID decodes instruction n + 1,
- EX executes instruction n and writes its result.

Term	Meaning in this CPU
Latency	Time for ONE instruction to go through all stages: 3 clock cycles
Throughput	Instructions finished per clock: 1 (when the pipeline is full)
Pipeline fill	The first instruction finishes in cycle 3; after that one finishes every cycle
N instructions (no jumps)	N + 2 clock cycles
Taken jump	2 extra clock cycles (2 instructions are flushed)

Table 4. Pipeline terms



3.4 Pipeline registers

Each stage must keep its own instruction while the stage before it is already working on the next one. This is done with pipeline registers placed between the stages. At every clock edge each pipeline register passes its contents one stage forward.

Pipeline register	Between	What it holds
IF/ID (the Instruction Register, IR)	IF and ID	The 12-bit instruction just fetched, its address, a valid bit
ID/EX	ID and EX	All control signals made by the decoder, plus Rd, Rs, imm, the address and a valid bit

Table 5. The two pipeline registers

3.5 Pipeline hazards and how this CPU handles them

A hazard is a situation where the overlap of instructions could give a wrong result. There are three kinds.

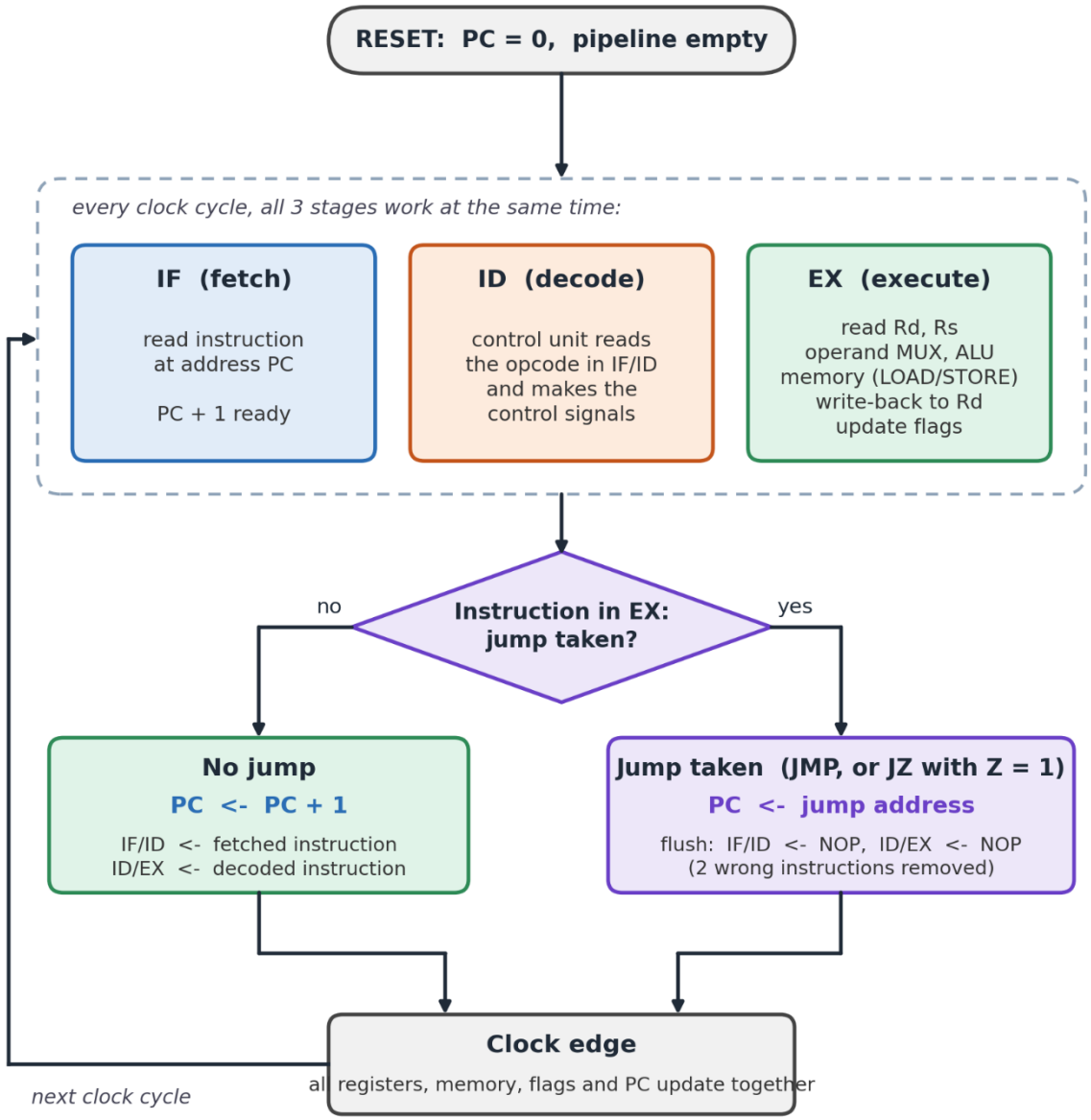
Hazard	What could go wrong	How this CPU avoids it
Structural	Two stages need the same hardware in the same clock	Separate instruction and data memories (Harvard); every block belongs to one stage
Data	An instruction reads a register before the previous instruction	Registers are read in EX, one clock after the previous

Hazard	What could go wrong	How this CPU avoids it
	has written it	instruction wrote them, so the value is always new (same idea as the classic ARM7 3-stage pipeline)
Control	After a jump, the instructions already fetched are the wrong ones	The jump is decided in EX; the 2 wrong instructions in IF/ID and ID/EX are flushed (turned into NOPs)

Table 6. Pipeline hazards

Data example. `LDI R1, 5` followed by `ADD R1, R2`. LDI writes R1 at the clock edge that ends its EX cycle. In the very next cycle ADD is in EX and reads R1, so it already sees 5. No waiting and no extra hardware is needed. The same holds for flags: SUB sets Zero at the end of its EX cycle and a following JZ tests it in the next cycle.

Control example. When `JMP 4` (at address 1) is in EX, the instructions at addresses 2 and 3 have already entered ID and IF. The branch unit sets `pc_sel = 1` and `flush = 1`: at the clock edge the PC loads 4 and both pipeline registers are emptied. Addresses 2 and 3 never execute. The cost is 2 empty cycles ("bubbles").



Result: one instruction can finish every clock cycle; a taken jump costs 2 extra cycles.

3.6 Relation to the single-cycle baseline in the project plan

The project plan describes a single-cycle baseline in which "one instruction completes per active clock cycle", and it also asks for a clocked Instruction Register loaded during fetch. A pure single-cycle CPU cannot have a real fetch register, because the instruction would arrive only after the clock edge that loads it. The 3-stage pipeline satisfies both points: there is a real fetch stage with a clocked IR, and once

the pipeline is full one instruction still completes every clock cycle. The only extra cost is 2 cycles at start-up and 2 cycles after each taken jump.

4. Digital Logic and Arithmetic Background

4.1 Combinational and sequential logic

- **Combinational logic** has no memory: its output depends only on its inputs right now. The ALU, the multiplexers, the decoder, the branch unit and the memory read paths are combinational. In Verilog: `assign` or `always @(*)`.
- **Sequential logic** remembers a value and changes only on a clock edge. The PC, the two pipeline registers, the register file, the flag register and the data-memory write are sequential. In Verilog: `always @(posedge clk)` with `<=`.

A pipelined CPU is a chain of sequential blocks (PC, IF/ID, ID/EX, registers) with combinational logic between them. At every rising clock edge all of them capture their new values at the same moment.

4.2 Combinational read, clocked write

The register file and data memory are read at any time (the output follows the address immediately) and written only on the rising clock edge. So `ADD R1, R2` can read R1, compute R1 + R2 and write the answer back into R1 in the same EX cycle: during the cycle the old value is used and the new value appears only after the edge.

4.3 Multiplexers

MUX	Input 0	Input 1	Select signal
Operand MUX	Register Rs value (qb)	Immediate imm[3:0]	`alu_src_sel`
Write-back MUX	ALU result	Data memory read data	`wb_sel`
PC MUX	PC + 1	Jump address (imm of the instruction in EX)	`pc_sel`

Table 7. The three datapath multiplexers (all 2-input, 4-bit)

4.4 Unsigned and signed (two's complement) numbers

Bits	Unsigned	Signed	Bits	Unsigned	Signed
0000	0	0	1000	8	-8
0001	1	+1	1001	9	-7
0010	2	+2	1010	10	-6
0011	3	+3	1011	11	-5
0100	4	+4	1100	12	-4

Bits	Unsigned	Signed	Bits	Unsigned	Signed
0101	5	+5	1101	13	-3
0110	6	+6	1110	14	-2
0111	7	+7	1111	15	-1

Table 8. 4-bit values read as unsigned and as two's complement

4.5 The adder/subtractor

Subtraction is done with an adder because A - B = A + (-B) = A + ~B + 1. In hardware every bit of B goes through an XOR gate controlled by `do_sub` (XOR with 1 flips a bit), and `do_sub` is also the carry-in that gives the "+1". This is exactly how `alu.v` is written in this project:

```
wire [3:0] b_eff = b ^ {4{do_sub}};                              // invert B when
subtracting
wire [4:0] total = {1'b0, a} + {1'b0, b_eff} + {4'b0, do_sub}; // total[4] = carry
```

4.6 Condition flags and the carry convention

Flag	Meaning	Updated by
Zero	The 4-bit result is 0000	ADD, SUB, AND, OR, XOR, NOT
Carry	ADD: carry out of bit 3. SUB: no-borrow (1 when A >= B)	ADD, SUB only

Table 9. Condition flags

Carry convention used everywhere in this project (ALU, flag register, testbenches and this report): for ADD, Carry = 1 means the answer did not fit in 4 bits; for SUB, Carry = 1 means **no borrow** (A is greater than or equal to B) and Carry = 0 means a borrow happened. This is the convention used by ARM processors.

Operation	Binary	Result	Zero	Carry	Meaning
5 + 3	0101 + 0011	1000	0	0	Fits in 4 bits
15 + 1	1111 + 0001	0000	1	1	Too big for 4 bits, result zero
5 - 3	0101 - 0011	0010	0	1	No borrow (5 >= 3)
3 - 5	0011 - 0101	1110	0	0	Borrow (3 < 5)
7 - 7	0111 - 0111	0000	1	1	Zero, no borrow

Table 10. Worked flag examples

5. CPU Components

Block	Stage	Inputs / connection	Role / output
Program Counter (PC)	IF	clock, reset, pc_en, next address	Address of the instruction to fetch
PC + 1 and PC MUX	IF	PC, jump address, pc_sel	Next address: PC + 1 or jump target
Instruction Memory	IF	PC address	12-bit instruction word
IF/ID register (IR)	IF to ID	instruction, address, flush	Holds the instruction being decoded
Control Unit: decoder	ID	opcode	All control signals for the instruction
ID/EX register	ID to EX	control signals, Rd, Rs, imm, flush	Holds the instruction being executed
Register File	EX	read addresses, write address / data, write enable	Two 4-bit operands; stores results
Operand MUX	EX	Rs value / immediate	ALU operand B
ALU	EX	two 4-bit operands, alu_ctrl	4-bit result, Carry, Zero
Flag Register	EX	ALU flags, write enables	Stored Zero and Carry
Data Memory	EX	address (from ALU), write data (Rd), mem_we	Read data for LOAD; store for STORE
Write-back MUX	EX	ALU result / memory data	Value written into Rd
Control Unit: branch unit	EX	is_jmp, is_jz, Zero	pc_sel and flush

Table 11. CPU blocks and the stage each belongs to

5.1 Program counter and next-address logic

The PC holds the address of the instruction being fetched. Because the pipeline fetches a new instruction every clock, the PC changes every clock: normally to PC + 1, or to the jump address when the branch unit says a jump is taken. With 4 bits a program can have at most 16 instructions; after address 15 the PC wraps to 0.

5.2 Instruction memory and the IF/ID register (IR)

The instruction memory is a table of sixteen 12-bit words loaded from a `.mem` text file. Its output goes into the IF/ID pipeline register, which is the CPU's Instruction Register. The IR also keeps the instruction's address and a valid bit (0 means the slot is empty, a "bubble").

5.3 Control unit (decoder) and the ID/EX register

In the ID stage the decoder turns the opcode into control signals. These signals, together with Rd, Rs and imm, are saved in the ID/EX register so the EX stage can use them in the next clock. When a jump is taken, ID/EX is loaded with all-zero control signals, so the flushed instruction changes nothing.

5.4 Register file

Four 4-bit registers with **two read ports** (qa, qb, read at any time) and **one write port** (clocked). Port A reads Rd and port B reads Rs. The CPU uses a **two-address** format, Rd = Rd op Rs: Rd is both the first source and the destination, which leaves room for a 4-bit immediate in the 12-bit word.

5.5 Operand multiplexer

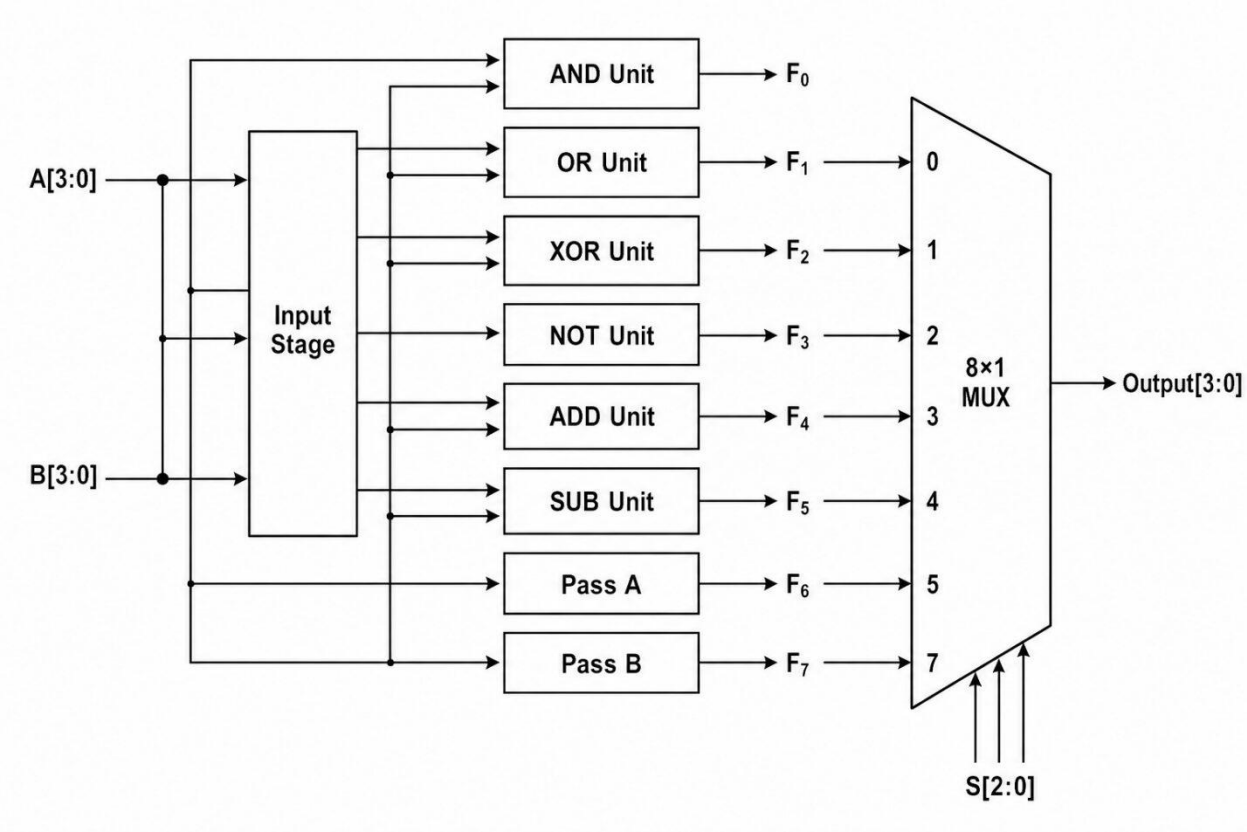
Chooses ALU input B: Rs for register-register instructions, or the 4-bit immediate for LDI, LOAD and STORE. Passing the immediate through the ALU lets constants and memory addresses use the same path as normal results.

5.6 Arithmetic logic unit

Purely combinational. A shared adder/subtractor does ADD and SUB, a logic unit does AND, OR, XOR and NOT, and a result MUX chooses the answer using `alu_ctrl`. A zero detector makes Zero. A **PASS** operation copies operand B to the output; it is used by LDI, MOV, LOAD and STORE.

alu_ctrl	Operation	result	Used by
0	ADD	a + b	ADD
1	SUB	a - b	SUB
2	AND	a & b	AND
3	OR	a | b	OR
4	XOR	a ^ b	XOR
5	NOT	~a	NOT
6	PASS	b	LDI, MOV, LOAD, STORE

Table 12. ALU control codes



5.7 Flag register, data memory and write-back MUX

The flag register stores Zero and Carry; they have separate write enables because logic instructions update Zero only. The 16 x 4-bit data memory takes its address from the ALU result, its write data from Rd, and is written only when `mem_we = 1`. The write-back MUX chooses whether Rd receives the ALU result or the memory data.

5.8 Branch unit

The branch unit looks at the instruction in EX. It sets `pc_sel = 1` for JMP, or for JZ when the stored Zero flag is 1. The same signal is used as `flush`, which empties the two pipeline registers.

5.9 Hardwired control

A control unit can be hardwired (signals made directly by logic gates) or microprogrammed (signals stored as microinstructions in a small memory). Hardwired control is smaller and faster and suits a small RISC CPU, as the project plan asks.

6. Instruction Set Architecture

6.1 Instruction format

Every instruction is exactly 12 bits long and uses the same layout, so the decoder always finds the same field in the same place. An 8-bit word would be too small to hold an opcode, two register numbers and an immediate.

```
bit:   11 10 9 8 | 7 6  | 5 4  | 3 2 1 0
       opcode(4) |Rd(2) |Rs(2) | imm / addr (4)

R-type  ADD R1, R2   : 0000 | 01 | 10 | 0000  = 0x060
I-type  LDI R3, 1010 : 0110 | 11 | 00 | 1010  = 0x6CA
J-type  JZ 9         : 1011 | 00 | 00 | 1001  = 0xB09
```

Bits	Field	Width	Meaning
[11:8]	opcode	4	Operation (16 codes; 12 used, 4 reserved)
[7:6]	Rd	2	Destination register and first source (R0-R3)
[5:4]	Rs	2	Second source register (R0-R3)
[3:0]	imm / addr	4	Immediate number, data-memory address or jump address

Table 13. Instruction fields

6.2 Instruction set

Opcode	Mnemonic	Operation / behaviour	Flags
0000	ADD Rd, Rs	Rd <- Rd + Rs	Zero, Carry
0001	SUB Rd, Rs	Rd <- Rd - Rs (Carry = no borrow)	Zero, Carry
0010	AND Rd, Rs	Rd <- Rd AND Rs	Zero
0011	OR Rd, Rs	Rd <- Rd OR Rs	Zero
0100	XOR Rd, Rs	Rd <- Rd XOR Rs	Zero
0101	NOT Rd	Rd <- bitwise NOT Rd	Zero
0110	LDI Rd, imm	Rd <- immediate[3:0]	-
0111	MOV Rd, Rs	Rd <- Rs	-

Opcode	Mnemonic	Operation / behaviour	Flags
1000	LOAD Rd, addr	Rd <- data_memory[addr]	-
1001	STORE Rd, addr	data_memory[addr] <- Rd	-
1010	JMP addr	PC <- addr	-
1011	JZ addr	If Zero = 1, PC <- addr; otherwise continue	-
1100-1111	(reserved)	No operation; free for future instructions	-

Table 14. Instruction set (as in the project plan)

6.3 Addressing modes

Mode	Where the operand comes from	Instructions
Register	A register named in the instruction	ADD, SUB, AND, OR, XOR, NOT, MOV
Immediate	A number written inside the instruction	LDI
Direct	A data-memory address inside the instruction	LOAD, STORE
Absolute jump	The jump address inside the instruction	JMP, JZ

Table 15. Addressing modes

6.4 Encoding an instruction

- `ADD R1, R2`: opcode 0000, Rd = R1 = 01, Rs = R2 = 10, imm = 0000, giving 0000 0110 0000 = **0x060**.
- `LDI R3, 1010`: 0110 11 00 1010 = **0x6CA**.
- `STORE R1, 1`: 1001 01 00 0001 = **0x941**.
- `JZ 9`: 1011 00 00 1001 = **0xB09**.

7. Control Unit Design

The control unit has two parts that work in different pipeline stages: the **instruction decoder** in the ID stage, which turns the opcode into control signals, and the **branch unit** in the EX stage, which decides whether a jump is taken and whether to flush.

7.1 Instruction decoder (ID stage)

The decoder is combinational logic written with **safe defaults**: every signal is first set to its "do nothing" value and the case statement only turns on the signals an instruction needs. This gives a fixed, known output for every opcode, avoids unwanted latches and makes the reserved opcodes act as NOP. In the implementation each opcode returns an **11-bit control word** in the order `{alu_ctrl[2:0], alu_src_sel, wb_sel, reg_we, mem_we, zero_we, carry_we, is_jmp, is_jz}`.

7.2 Control signals

Signal	Made in	Used in	Meaning
alu_ctrl (3 bits)	ID	EX	ALU operation
alu_src_sel	ID	EX	Operand MUX: 0 = Rs, 1 = immediate
wb_sel	ID	EX	Write-back MUX: 0 = ALU result, 1 = memory data
reg_we	ID	EX	Write the destination register Rd
mem_we	ID	EX	Write the data memory
zero_we, carry_we	ID	EX	Update the Zero / Carry flag
is_jmp, is_jz	ID	EX	Tell the branch unit what kind of jump this is
pc_sel	EX (branch unit)	IF	PC MUX: 0 = PC + 1, 1 = jump address
flush	EX (branch unit)	IF/ID, ID/EX	Empty both pipeline registers
pc_en, ir_load	fixed at 1	IF	Fetch a new instruction every clock

Table 16. Control signals

7.3 Decoder truth table

Instr	alu_ctrl	alu_src	wb_sel	reg_we	mem_we	zero_we	carry_we	is_jmp	is_jz
ADD	ADD	0	0	1	0	1	1	0	0

Instr	alu_ctrl	alu_src	wb_sel	reg_we	mem_we	zero_we	carry_we	is_jmp	is_jz
SUB	SUB	0	0	1	0	1	1	0	0
AND	AND	0	0	1	0	1	0	0	0
OR	OR	0	0	1	0	1	0	0	0
XOR	XOR	0	0	1	0	1	0	0	0
NOT	NOT	0	0	1	0	1	0	0	0
LDI	PASS	1	0	1	0	0	0	0	0
MOV	PASS	0	0	1	0	0	0	0	0
LOAD	PASS	1	1	1	0	0	0	0	0
STORE	PASS	1	0	0	1	0	0	0	0
JMP	-	0	0	0	0	0	0	1	0
JZ	-	0	0	0	0	0	0	0	1
reserved	-	0	0	0	0	0	0	0	0

Table 17. Decoder outputs for every opcode (checked by tb_control_unit)

7.4 Branch unit (EX stage)

Instruction in EX	Zero flag	pc_sel	flush	What happens at the clock edge
not a jump	any	0	0	PC <- PC + 1; pipeline moves forward normally
JMP	any	1	1	PC <- jump address; IF/ID and ID/EX emptied
JZ	0	0	0	Jump not taken; continue normally
JZ	1	1	1	PC <- jump address; IF/ID and ID/EX emptied

Table 18. Branch unit truth table (checked by tb_control_unit)

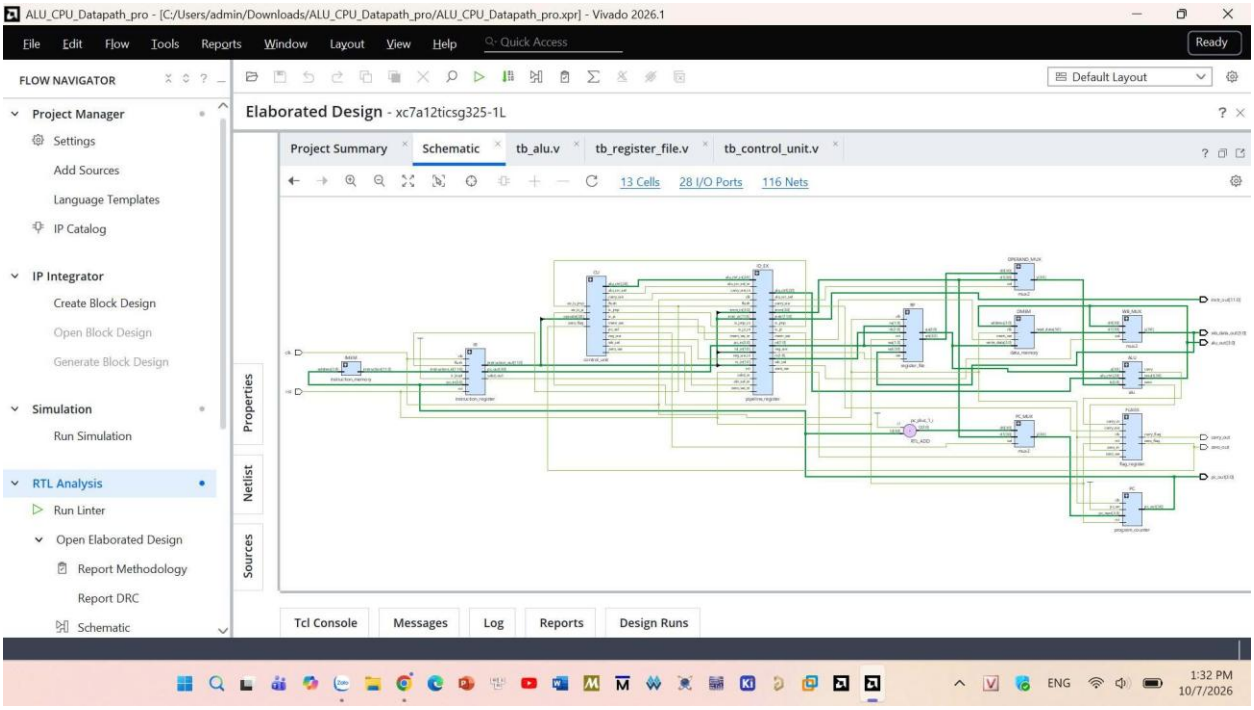
A conditional jump is just a multiplexer: for JZ, `pc_sel` is simply the stored Zero flag. If it is 1 the PC MUX picks the jump address, otherwise PC + 1. Implemented as `pc_sel = is_jmp | (is_jz & zero_flag)` and `flush = pc_sel`.

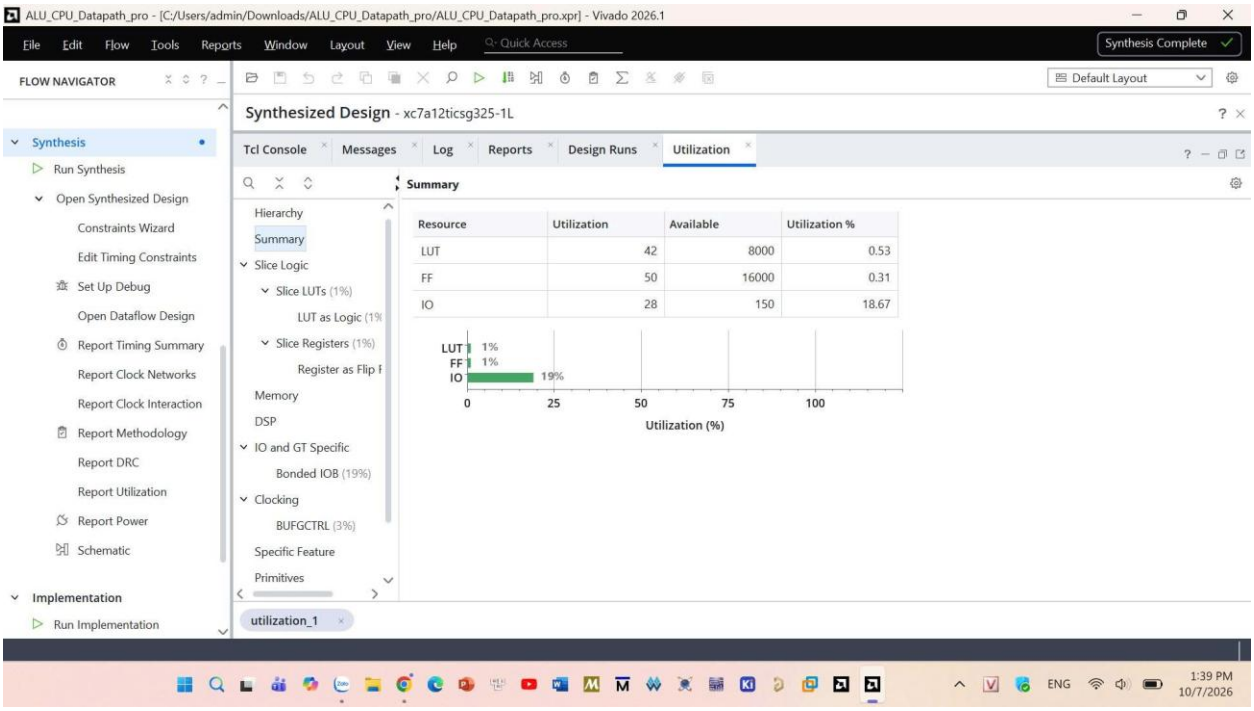
8. Datapath Schematic and Operation

The CPU is divided into its three pipeline stages. Solid lines carry data; dashed arrows are control signals.

```
STAGE 1 - IF                STAGE 2 - ID           STAGE 3 - EX

PC MUX -> PC -> IMEM -> [IF/ID IR] -> Decoder -> [ID/EX] -> RegFile (Rd, Rs)
  ^         |                                                    |
  |       PC + 1                                          OP MUX (Rs / imm)
  |                                                             |
  |                                                          ALU --> Flags (Z, C)
  |                                                             |
  |                                                   DMEM (addr = ALU, wd = Rd)
  |                                                             |
  |                                                        WB MUX (ALU / mem)
  |                                                             |
  |                                          write-back into Rd <----+
  +---- pc_sel <---- Branch unit (EX) --- flush (empties IF/ID and ID/EX)
```





Path	Route	Width
Instruction path	PC -> instruction memory -> IF/ID (IR)	4-bit address, 12-bit instruction
Control path	IF/ID opcode -> decoder -> ID/EX -> MUX selects, ALU, register / memory write enables	4-bit opcode, control lines
Data path	Register file -> operand MUX -> ALU -> write-back MUX -> register file	4 bits
Memory path	ALU result -> data-memory address; Rd -> write data; read data -> write-back MUX	4 bits
Jump path	ID/EX imm -> PC MUX -> PC; branch unit -> pc_sel and flush	4 bits + control

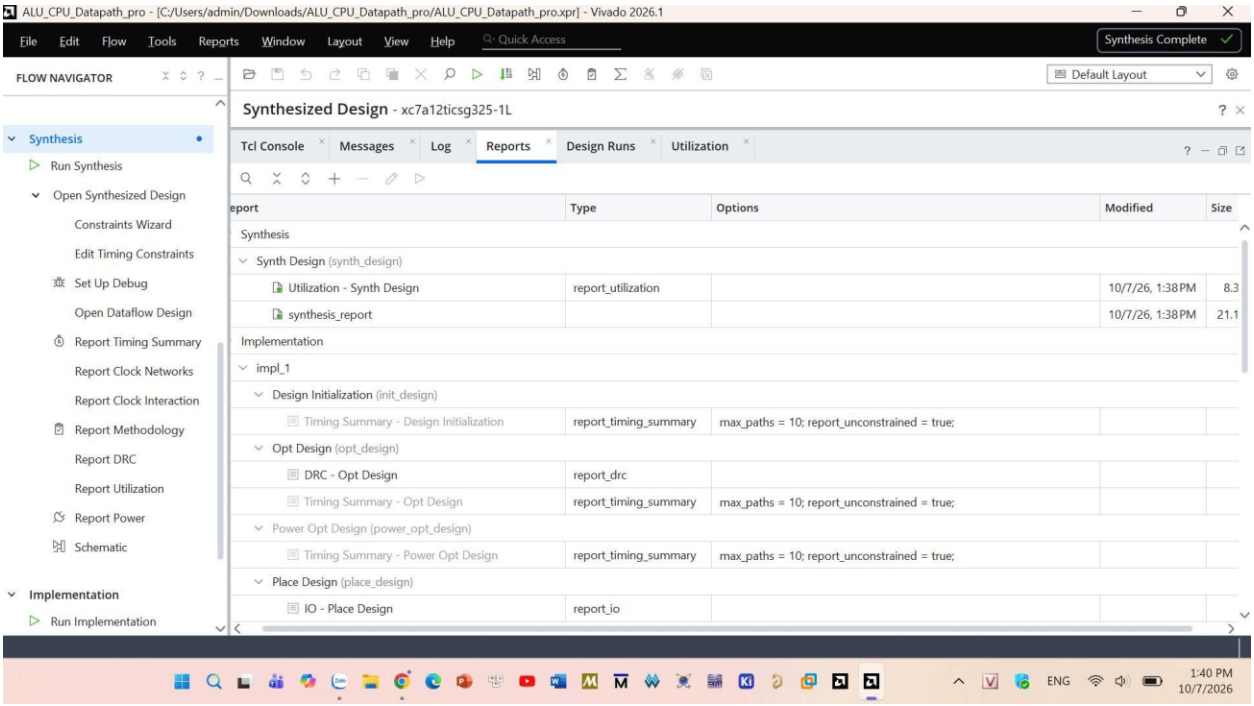
Table 19. Block-level paths (project plan Section 3.2)

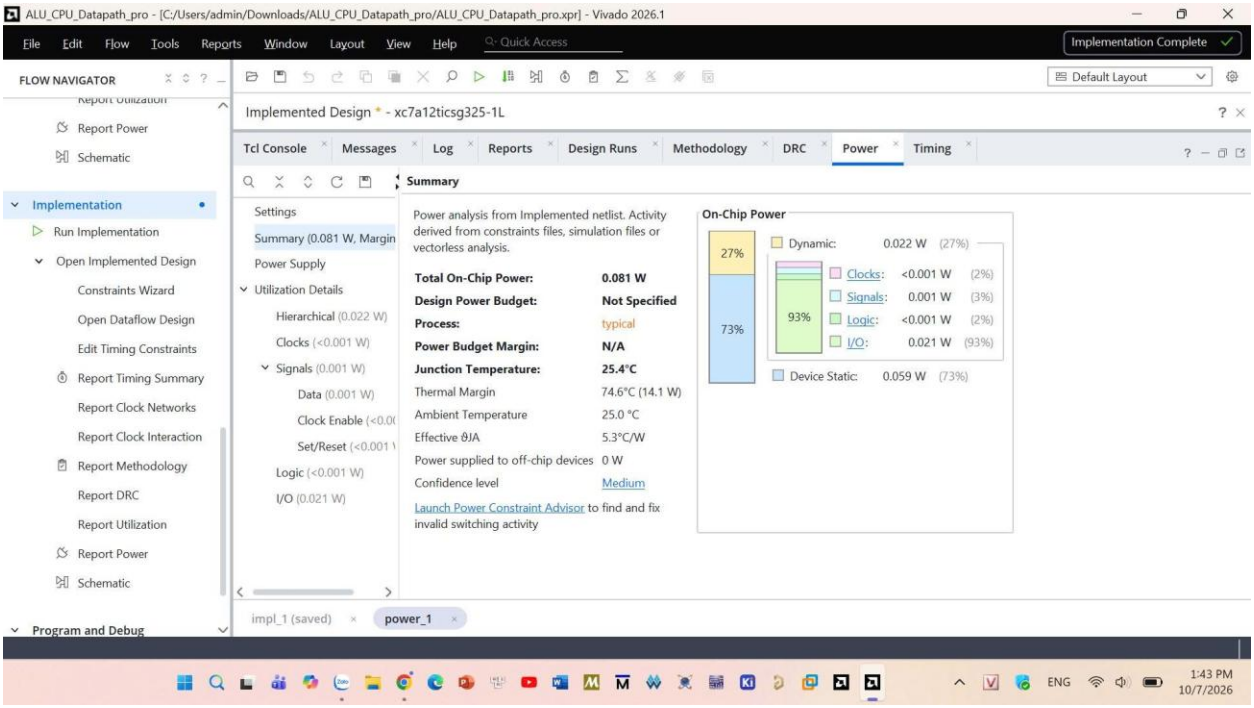
Operation of each instruction type

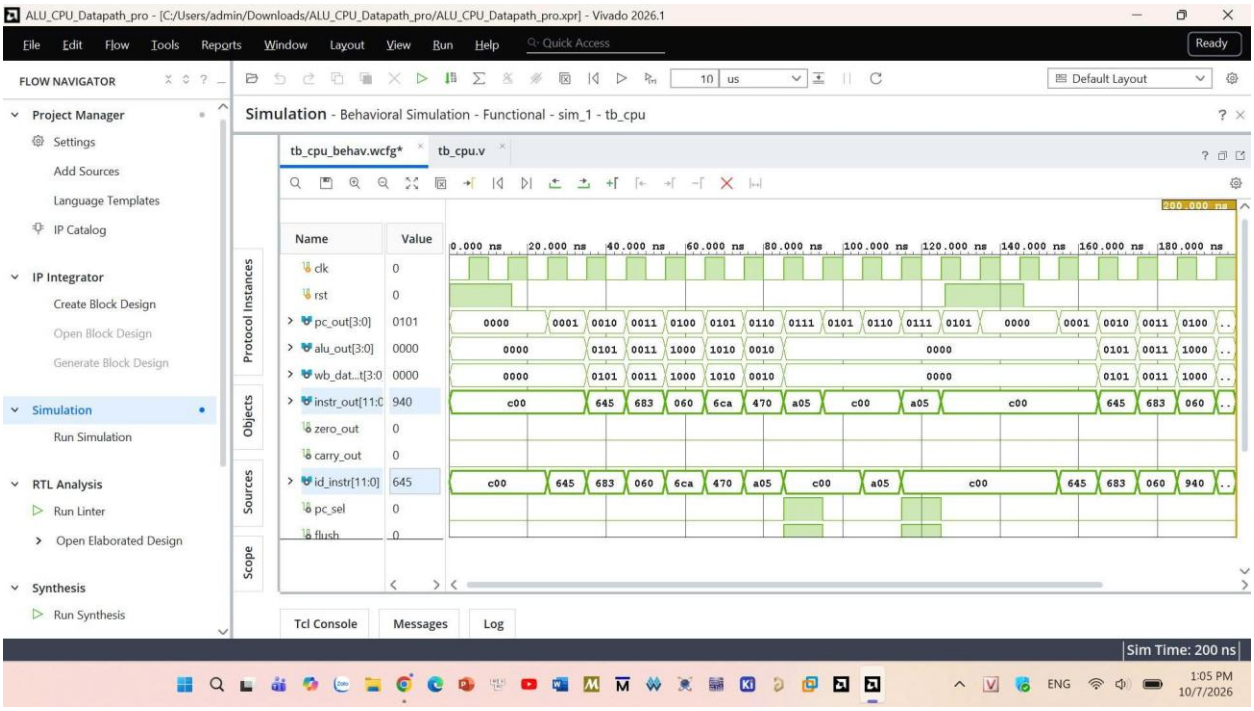
Instruction	IF (clock 1)	ID (clock 2)	EX (clock 3)	Result at end of clock 3

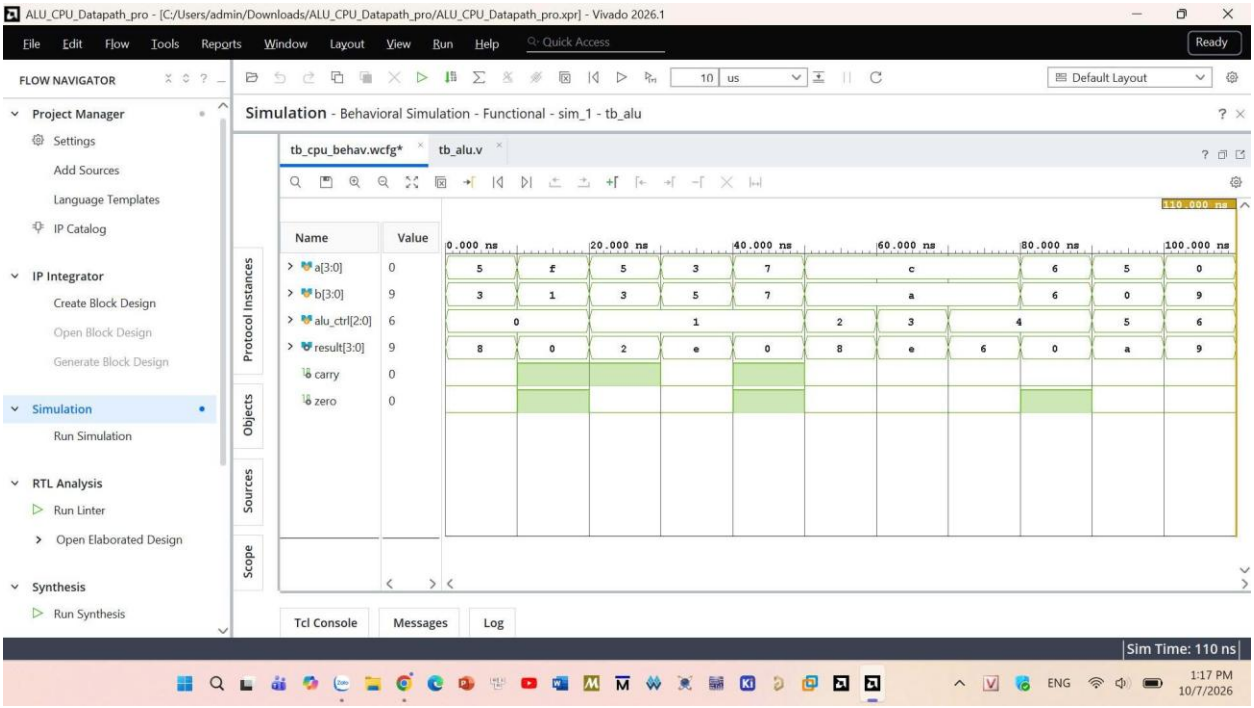
Instruction	IF (clock 1)	ID (clock 2)	EX (clock 3)	Result at end of clock 3
ADD Rd, Rs	Read instruction at PC, save in IF/ID	alu_ctrl = ADD, reg_we = 1, zero_we = carry_we = 1; save in ID/EX	Read Rd, Rs; operand MUX passes Rs; ALU adds; wb_sel = 0	Rd <- sum; Zero, Carry <- flags
LDI Rd, imm	same	alu_ctrl = PASS, alu_src_sel = 1, reg_we = 1	Operand MUX passes the immediate; ALU copies it	Rd <- imm; flags unchanged
LOAD Rd, addr	same	PASS, alu_src_sel = 1, wb_sel = 1, reg_we = 1	Immediate goes through the ALU to the memory address; memory gives the value; wb_sel = 1	Rd <- data_memory[addr]
STORE Rd, addr	same	PASS, alu_src_sel = 1, mem_we = 1	Rd goes to the memory write data; immediate through the ALU to the address	data_memory[addr] <- Rd; no register changes
JZ addr (taken)	same	is_jz = 1	Branch unit reads Zero = 1, sets pc_sel = flush = 1	PC <- jump address; IF/ID and ID/EX emptied; if Zero = 0 nothing special happens

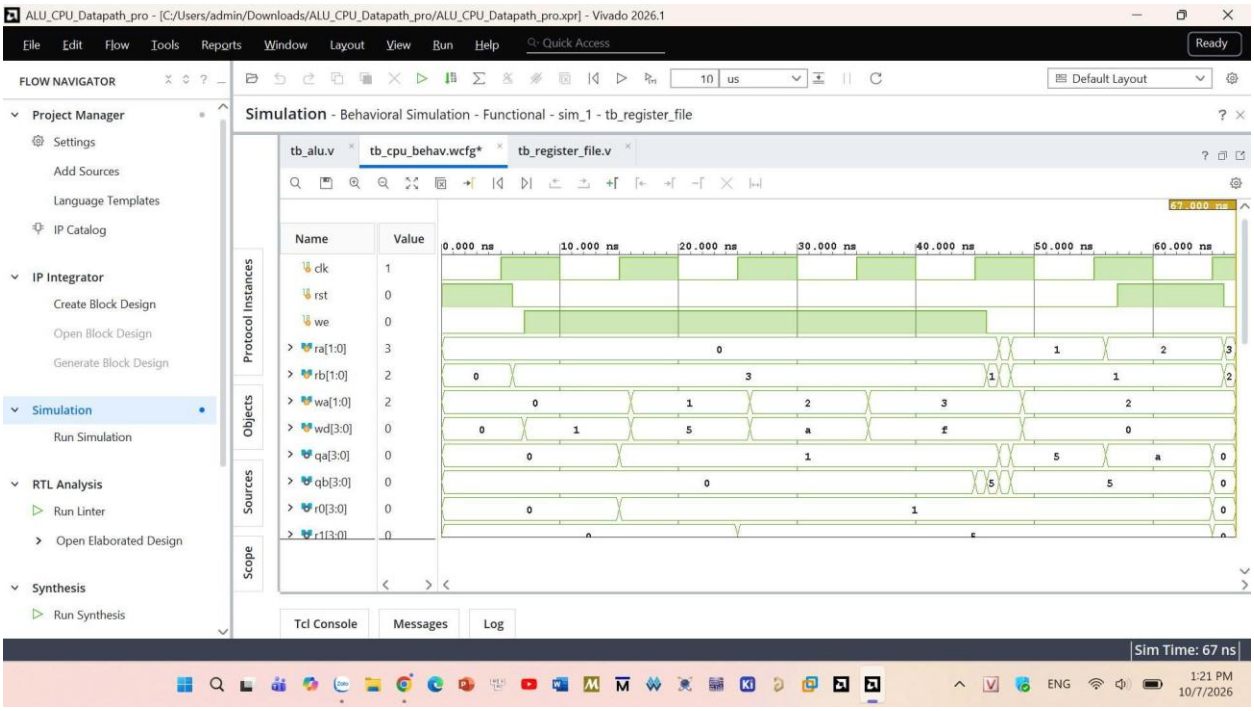
Table 20. Path of each instruction through the three stages

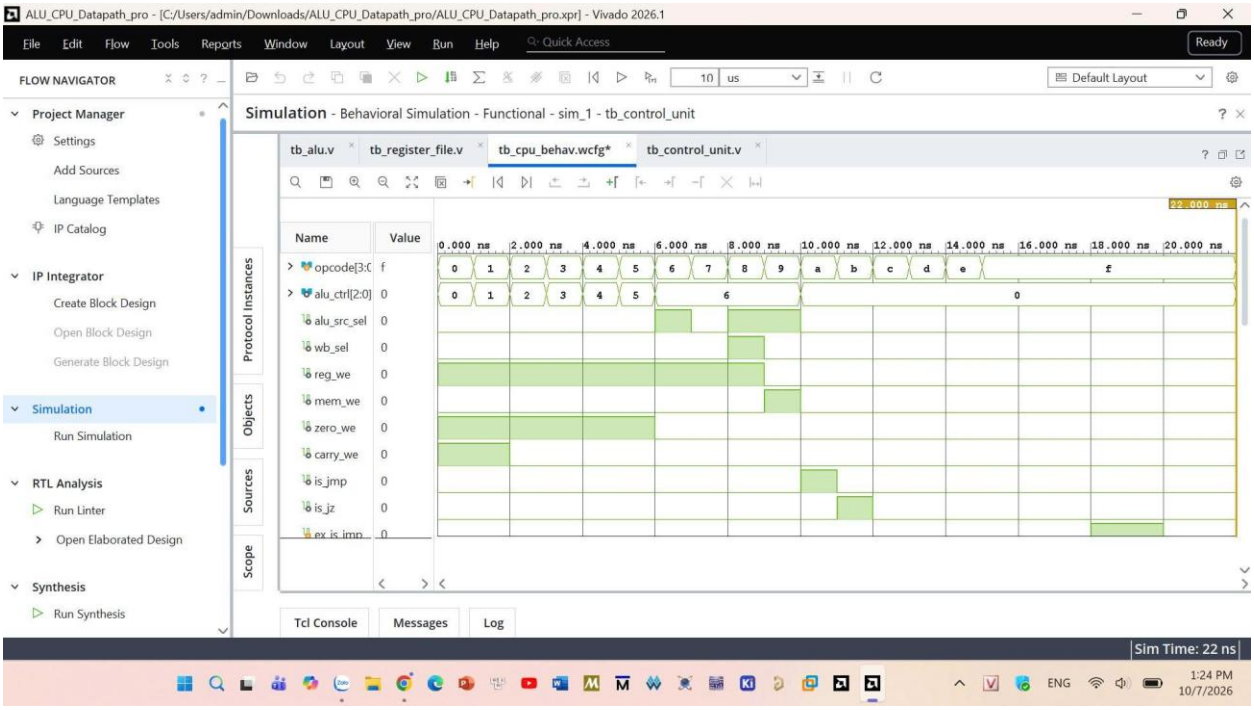

ALU_CPU_Datapath_pro - [C:/Users/admin/Downloads/ALU_CPU_Datapath_pro/ALU_CPU_Datapath_pro.xpr] - Vivado 2026.1
File Edit Flow Tools Reports Window Layout View Help
Quick Access
Synthesis Complete
FLOW NAVIGATOR
Default Layout
Synthesized Design - xc7a12ticsg325-1L
Tcl Console Messages Log Reports Design Runs Utilization
Synthesis
Run Synthesis
Open Synthesized Design
Constraints Wizard
Edit Timing Constraints
Set Up Debug
Open Dataflow Design
Report Timing Summary
Report Clock Networks
Report Clock Interaction
Report Methodology
Report DRC
Report Utilization
Report Power
Schematic
Implementation
Run Implementation
Report Type Options Modified Size
Synthesis
Synth Design (synth_design)
Utilization - Synth Design report_utilization 10/7/26, 1:38 PM 8.3
synthesis_report 10/7/26, 1:38 PM 21.1
Implementation
impl_1
Design Initialization (init_design)
Timing Summary - Design Initialization report_timing_summary max_paths = 10; report_unconstrained = true;
Opt Design (opt_design)
DRC - Opt Design report_drc
Timing Summary - Opt Design report_timing_summary max_paths = 10; report_unconstrained = true;
Power Opt Design (power_opt_design)
Timing Summary - Power Opt Design report_timing_summary max_paths = 10; report_unconstrained = true;
Place Design (place_design)
IO - Place Design report_io
ENG
1:40 PM
10/7/2026


ALU_CPU_Datapath_pro - [C:/Users/admin/Downloads/ALU_CPU_Datapath_pro/ALU_CPU_Datapath_pro.xpr] - Vivado 2026.1
File Edit Flow Tools Reports Window Layout View Help
Quick Access
Implementation Complete
FLOW NAVIGATOR
Default Layout
Report Power
Schematic
Implementation
Run Implementation
Open Implemented Design
Constraints Wizard
Open Dataflow Design
Edit Timing Constraints
Report Timing Summary
Report Clock Networks
Report Clock Interaction
Report Methodology
Report DRC
Report Utilization
Report Power
Schematic
Program and Debug
Implemented Design * - xc7a12ticsg325-1L
Tcl Console Messages Log Reports Design Runs Methodology DRC Power Timing
Summary
Settings
Summary (0.081 W, Margin
Power Supply
Utilization Details
Hierarchical (0.022 W)
Clocks (<0.001 W)
Signals (0.001 W)
Data (0.001 W)
Clock Enable (<0.0
Set/Reset (<0.001
Logic (<0.001 W)
I/O (0.021 W)
Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.
Total On-Chip Power: 0.081 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 25.4°C
Thermal Margin 74.6°C (14.1 W)
Ambient Temperature 25.0 °C
Effective θJA 5.3°C/W
Power supplied to off-chip devices 0 W
Confidence level Medium
Launch Power Constraint Advisor to find and fix invalid switching activity
On-Chip Power
27%
73%
93%
Dynamic: 0.022 W (27%)
Clocks: <0.001 W (2%)
Signals: 0.001 W (3%)
Logic: <0.001 W (2%)
I/O: 0.021 W (93%)
Device Static: 0.059 W (73%)
impl_1 (saved)
power_1
ENG
1:43 PM
10/7/2026


ALU_CPU_Datapath_pro - [C:/Users/admin/Downloads/ALU_CPU_Datapath_pro/ALU_CPU_Datapath_pro.xpr] - Vivado 2026.1
File Edit Flow Tools Reports Window Layout View Run Help
Quick Access
Ready
FLOW NAVIGATOR
Project Manager
Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Run Simulation
RTL Analysis
Run Linter
Open Elaborated Design
Synthesis
Run Synthesis
Default Layout
Simulation - Behavioral Simulation - Functional - sim_1 - tb_cpu
tb_cpu_behav.wcfg*
tb_cpu.v
Name
Value
clk
rst
pc_out[3:0]
alu_out[3:0]
wb_dat...t[3:0]
instr_out[11:0]
zero_out
carry_out
id_instr[11:0]
pc_sel
flush
Protocol Instances
Objects
Sources
Scope
Tcl Console
Messages
Log
Sim Time: 200 ns
ENG
1:05 PM
10/7/2026


ALU_CPU_Datapath_pro - [C:/Users/admin/Downloads/ALU_CPU_Datapath_pro/ALU_CPU_Datapath_pro.xpr] - Vivado 2026.1
File Edit Flow Tools Reports Window Layout View Run Help
Quick Access
Ready
FLOW NAVIGATOR
Project Manager
Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Run Simulation
RTL Analysis
Run Linter
Open Elaborated Design
Synthesis
Run Synthesis
Default Layout
Simulation - Behavioral Simulation - Functional - sim_1 - tb_alu
tb_cpu_behav.wcfg*
tb_alu.v
Name
Value
a[3:0]
b[3:0]
alu_ctrl[2:0]
result[3:0]
carry
zero
Protocol Instances
Objects
Sources
Scope
Tcl Console
Messages
Log
Sim Time: 110 ns
ENG
1:17 PM
10/7/2026





Part B. Practical Implementation

9. Project Plan and Compliance

This chapter checks the design against the project plan given by the project seniors.

9.1 Requirements traceability

Plan requirement (Section 5)	How it is met	Verified by
Synchronous operation using a common clock	All sequential blocks use the same `clk`, rising edge	All testbenches
Reset places PC and control state in a known condition	PC = 0, registers = 0, flags = 0, both pipeline registers empty (NOP)	tb_cpu Test 5, tb_register_file
Register file: two combinational read ports, one clocked write port	`register_file.v`: qa and qb by `assign`, write on posedge clk	tb_register_file
ALU supports ADD, SUB, AND, OR, XOR, NOT	`alu.v` (adder_subtractor + logic_unit)	tb_alu, tb_cpu Tests 2-3
ALU generates Zero and a Carry / borrow flag	zero_detector; carry = carry-out / no-borrow	tb_alu, tb_cpu Test 2
Control unit decodes every opcode deterministically	instruction_decoder with safe defaults; reserved opcodes = NOP	tb_control_unit (all 16 opcodes)
PC increments and supports jump / conditional branch	PC MUX + branch unit (JMP, JZ) + flush	tb_control_unit, tb_cpu Test 4
CPU executes a test program and produces expected results	4 programs (.mem files)	tb_cpu Tests 1-4

Table 21. Functional requirements and where they are met

9.2 Module plan and interfaces

Module / file	Plan interface	Implemented ports
alu.v	a, b, alu_ctrl, result, carry, zero	same
register_file.v	clk, rst, ra, rb, wa, wd, we, qa, qb	same
program_counter.v	clk, rst, pc_en, pc_next, pc_out	same
instruction_memory.v	address, instruction	same
instruction_register.v	clk, rst, ir_load, instruction_in, instruction_out	same + flush, pc_in, pc_out, valid_out (pipeline)
control_unit.v	opcode, flags, alu_ctrl, reg_we, mux_sel, pc controls	opcode, zero_flag, alu_ctrl, reg_we, mem_we, alu_src_sel, wb_sel, zero_we, carry_we,

Module / file	Plan interface	Implemented ports
		is_jmp, is_jz, pc_sel, flush
data_memory.v	clk, address, write_data, mem_we, read_data	same
cpu_top.v	clk, rst, observable debug outputs	clk, rst, pc_out, instr_out, alu_out, wb_data_out, zero_out, carry_out
tb_cpu.v	stimulus, checks, pass/fail reporting	5 tests, 44 checks, PASS/FAIL summary

Table 22. Module interfaces: project plan versus implementation

Extra files not listed separately in the plan: `pipeline_register.v` (the ID/EX register), `mux.v` (the three multiplexers), `flag_register.v` and three module-level testbenches.

9.3 Design decisions and differences from the plan

Topic	Plan	This implementation	Reason
Execution model	Single-cycle baseline	3-stage pipeline (IF, ID, EX)	Real fetch into a clocked IR and a separate decode stage, while still finishing one instruction per clock (section 3.6)
Branch handling	PC loads branch target	Decided in EX; 2 instructions flushed	Simple and always correct; costs 2 cycles per taken jump
Data memory	Optional	Included	Lets LOAD and STORE be built and tested
Carry after SUB	Must be documented	Carry = 1 means no borrow	Same convention in ALU, flags and tests
Memory address	"ALU address"	ALU passes the immediate to the address	Same path as LDI
Reserved opcodes	Unused	Behave as NOP	Known decode result for every opcode

Table 23. Design decisions

9.4 Two-week schedule

Week	Work package	Activities	Milestone / evidence
1	Requirements and ISA	Scope, instruction format, opcode table, reset, flags, test strategy	Chapters 1, 6
1	Datapath design	Block diagram; buses, register file, ALU, MUX, PC, memories, pipeline registers	Datapath schematic (chapter 8), interface table
1	ALU and registers	Code ALU and register file; module-level testbenches	tb_alu and tb_register_file pass
2	Control and fetch path	PC, instruction memory, IR, decoder, branch unit	tb_control_unit passes
2	Integration	Connect stages in cpu_top; pipeline registers; data memory; run programs	tb_cpu Tests 1-4 pass
2	Verification and documentation	Vivado simulation, waveforms, pipeline charts, report	88 / 88 checks; this report

Table 24. Project schedule

9.5 Risks and mitigation

Risk	Possible effect	What was done
Instruction fields do not fit	Unclear decoding	12-bit format fixed before coding
Control / datapath mismatch	Wrong operation or write	Decoder truth table checked opcode by opcode
Carry / borrow ambiguity	Inconsistent flags	One convention (no-borrow) everywhere
Pipeline hazards	Wrong value read, or wrong instruction executed after a jump	Register read in EX (no data hazard); flush after a taken jump; both tested in tb_cpu

Risk	Possible effect	What was done
Uninitialised memory / registers	X values in simulation	Reset; memories initialised; X check in tb_cpu
Memory file not found by the tool	Empty program or wrong results	Add `program.mem` to Design Sources (see section 15.5)
Scope expansion	Running out of time	FPGA demo and extra branches kept optional

Table 25. Risks and mitigation

10. Project Structure and Coding Style

10.1 Tools

- **Xilinx Vivado 2026.1** with the Vivado Simulator (XSim) for all behavioural simulation, schematics and optional synthesis. Project: `ALU_CPU_Datapath_pro`, device Artix-7 `xc7a12ticsg325-1L`, target language Verilog.
- **Icarus Verilog 12.0** was used to pre-check every testbench before moving to Vivado; the files run unchanged in Vivado and ModelSim / Questa (plain Verilog-2005, no SystemVerilog).
- Any text editor for the Verilog and `.mem` files.

10.2 Coding style

- **One job per module.** Larger blocks are split into small sub-modules in the same file: `alu.v` has adder_subtractor, logic_unit and zero_detector; `control_unit.v` has instruction_decoder and branch_unit.
- **Plain, readable constructs:** `assign`, `case` and `if` only; the decoder returns one packed control word per opcode; the pipeline register moves signals as concatenations.
- **Every register has a defined start value** (`initial` blocks) so no X appears before the first reset.
- **A comment on the purpose of each block**, and a header in every file that lists its interface.
- **`cpu_top.v` is written stage by stage:** STAGE 1 IF, STAGE 2 ID, STAGE 3 EX, with signal names starting with `id_` or `ex_` to show which stage they belong to.

10.3 Module hierarchy and files

```
tb_cpu                      (CPU-level test)      tb_alu, tb_register_file,
tb_control_unit (module tests)
  cpu_top
    program_counter  instruction_memory  instruction_register (IF/ID)
    control_unit  [instruction_decoder, branch_unit]
    pipeline_register (ID/EX)  register_file
    alu  [adder_subtractor, logic_unit, zero_detector]
    flag_register  data_memory  mux2 (x3: PC, operand, write-back)
```

File	Contents
alu.v	adder_subtractor, logic_unit, zero_detector, alu

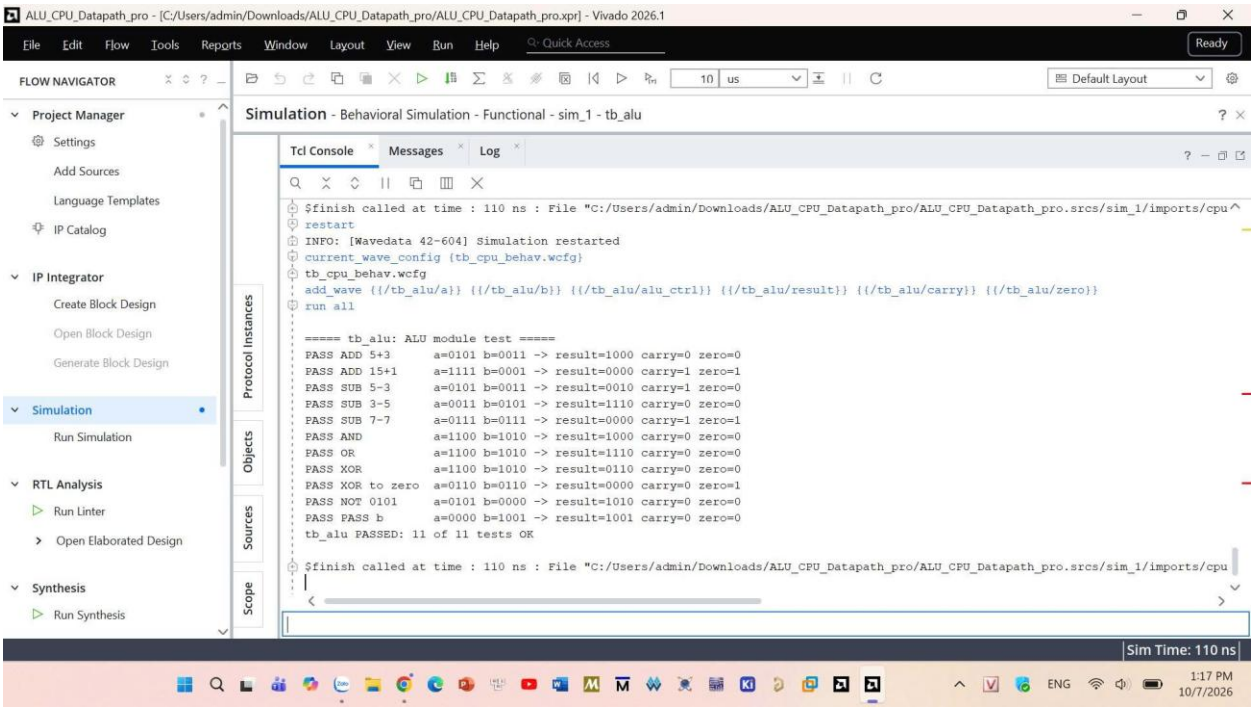
File	Contents
register_file.v	register_file (R0-R3)
program_counter.v	program_counter
instruction_memory.v	instruction_memory (16 x 12, loads a .mem file)
instruction_register.v	instruction_register = IF/ID pipeline register
pipeline_register.v	pipeline_register = ID/EX pipeline register
control_unit.v	instruction_decoder, branch_unit, control_unit
data_memory.v	data_memory (16 x 4)
mux.v	mux2 (used 3 times)
flag_register.v	flag_register (Zero, Carry)
cpu_top.v	cpu_top: connects everything, stage by stage
tb_alu.v, tb_register_file.v, tb_control_unit.v	module-level testbenches
tb_cpu.v	CPU-level testbench
program.mem	demonstration program (plan Appendix A)
prog_arith.mem, prog_logic.mem, prog_jump.mem	test programs used by tb_cpu

Table 26. Project files (19 files: 11 design, 4 testbench, 4 memory)

11. Module Implementation

This chapter gives the Verilog code of every design module, each followed by a short explanation. The testbenches are described in chapter 13; their full source is in the project folder.

11.1 ALU (alu.v)




```
// FILE: alu.v - 4-bit ALU (purely combinational)
// alu_ctrl: 0 ADD | 1 SUB | 2 AND | 3 OR | 4 XOR | 5 NOT | 6 PASS(b)
// CARRY: ADD = carry-out; SUB = 1 when a >= b (no borrow); logic/PASS = 0

module adder_subtractor (
    input  [3:0] a,
    input  [3:0] b,
    input        do_sub,      // 0 = add, 1 = subtract
    output [3:0] result,
    output       carry
);
    wire [3:0] b_eff = b ^ {4{do_sub}};                          // invert B for
SUB
    wire [4:0] total = {1'b0, a} + {1'b0, b_eff} + {4'b0, do_sub}; // do_sub = carry-
in
    assign result = total[3:0];
    assign carry  = total[4];                                     // carry-out /
no-borrow
endmodule

module logic_unit (
    input      [3:0] a,
    input      [3:0] b,
    input      [2:0] alu_ctrl,
    output reg [3:0] result
);
    always @(*) begin
        case (alu_ctrl)
            3'd2:    result = a & b;   // AND
            3'd3:    result = a | b;   // OR
```

```
            3'd4:    result = a ^ b;   // XOR
            3'd5:    result = ~a;      // NOT (uses A only)
            3'd6:    result = b;       // PASS B
            default: result = 4'b0000;
        endcase
    end
endmodule

module zero_detector (
    input  [3:0] value,
    output       zero
);
    assign zero = ~(|value);           // 1 only when all four bits are 0
endmodule

module alu (
    input  [3:0] a,
    input  [3:0] b,
    input  [2:0] alu_ctrl,
    output [3:0] result,
    output       carry,
    output       zero
);
    wire [3:0] ar_res, lg_res;
    wire       ar_cy;

    wire is_sub   = (alu_ctrl == 3'd1);
    wire is_arith = (alu_ctrl == 3'd0) | is_sub;

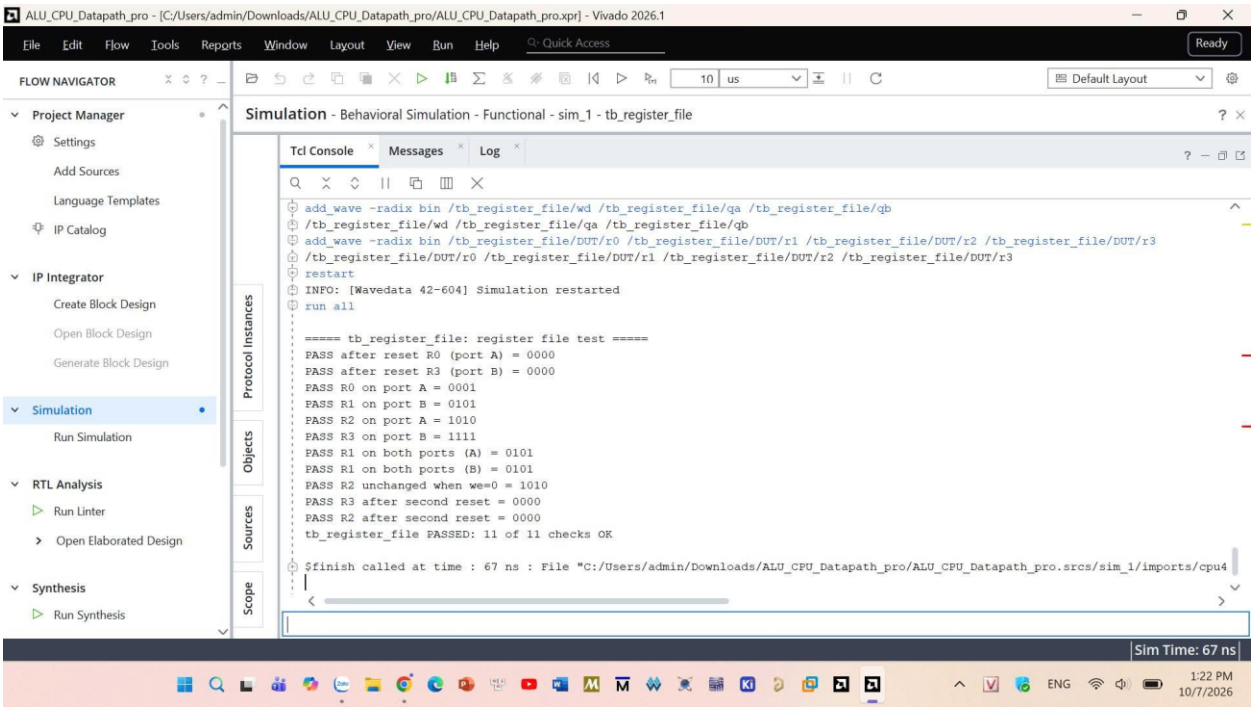
    adder_subtractor U_ARITH (.a(a), .b(b), .do_sub(is_sub),
                              .result(ar_res), .carry(ar_cy));
    logic_unit       U_LOGIC (.a(a), .b(b), .alu_ctrl(alu_ctrl),
                              .result(lg_res));

    assign result = is_arith ? ar_res : lg_res;
    assign carry  = is_arith ? ar_cy  : 1'b0;

    zero_detector    U_ZERO  (.value(result), .zero(zero));
endmodule
```

- **adder_subtractor:** B is XOR-ed with `do_sub`, and `do_sub` is also the carry-in, so one adder does both ADD and SUB. The 5th bit of the sum is the carry (for SUB it equals 1 exactly when a >= b).
- **logic_unit:** one `case` for AND, OR, XOR, NOT and PASS.
- **zero_detector:** reduction NOR of the four result bits.
- **alu:** picks the adder result for ADD/SUB and the logic result otherwise. Every output is driven in every case, so no latches are created.

11.2 Register file (register_file.v)




```
module register_file (
    input        clk,
    input        rst,
    input  [1:0] ra,      // read address, port A
    input  [1:0] rb,      // read address, port B
    input  [1:0] wa,      // write address
    input  [3:0] wd,      // write data
    input        we,      // write enable
    output [3:0] qa,      // read data, port A
    output [3:0] qb       // read data, port B
);
    reg [3:0] regs [0:3];
    integer n;

    initial for (n = 0; n < 4; n = n + 1) regs[n] = 4'b0000;   // no X at power-up

    always @(posedge clk) begin                                  // synchronous write
/ reset
        if (rst) begin
            for (n = 0; n < 4; n = n + 1) regs[n] <= 4'b0000;
        end else if (we) begin
            regs[wa] <= wd;
        end
    end

    assign qa = regs[ra];                                        // combinational
reads
    assign qb = regs[rb];

    wire [3:0] r0 = regs[0];                                     // names for the
```

```
waveform
    wire [3:0] r1 = regs[1];
    wire [3:0] r2 = regs[2];
    wire [3:0] r3 = regs[3];
endmodule
```

The four registers are an array. Writing is clocked and uses the address `wa`; the two read ports are plain `assign` statements, so `qa` and `qb` follow `ra` and `rb` immediately. The wires `r0` to `r3` only exist so the registers are easy to add to a waveform.

11.3 Program counter (program_counter.v)

```
module program_counter (
    input            clk,
    input            rst,
    input            pc_en,
    input      [3:0] pc_next,
    output reg [3:0] pc_out
);
    initial pc_out = 4'd0;

    always @(posedge clk) begin
        if (rst)        pc_out <= 4'd0;
        else if (pc_en) pc_out <= pc_next;
    end
endmodule
```

11.4 Instruction memory (instruction_memory.v)

```
module instruction_memory #(
    parameter PROGRAM_FILE = "program.mem"
)(
    input  [3:0]  address,
    output [11:0] instruction
);
    reg [11:0] mem [0:15];
    integer j;

    initial begin
        for (j = 0; j < 16; j = j + 1) mem[j] = 12'hC00;   // C00 = reserved opcode =
NOP
        $readmemh(PROGRAM_FILE, mem);                       // one hex word per line
    end

    assign instruction = mem[address];
endmodule
```

`PROGRAM_FILE` is a parameter, so the same hardware runs different programs. All words are first set to `C00` (a reserved opcode, so a NOP) and the file then overwrites the used words. Empty words therefore never hold unknown values.

11.5 IF/ID pipeline register (instruction_register.v)

```
module instruction_register (
    input             clk,
    input             rst,
```

```
    input             ir_load,
    input             flush,
    input      [11:0] instruction_in,
    input      [3:0]  pc_in,
    output reg [11:0] instruction_out,
    output reg [3:0]  pc_out,
    output reg        valid_out
);
    localparam [11:0] NOP = 12'hC00;

    initial begin
        instruction_out = NOP;
        pc_out          = 4'd0;
        valid_out       = 1'b0;
    end

    always @(posedge clk) begin
        if (rst | flush) begin
            instruction_out <= NOP;
            pc_out          <= 4'd0;
            valid_out       <= 1'b0;       // empty slot (bubble)
        end else if (ir_load) begin
            instruction_out <= instruction_in;
            pc_out          <= pc_in;
            valid_out       <= 1'b1;
        end
    end
endmodule
```

Every clock it saves the instruction just fetched, its address and `valid = 1`. On reset or flush it is loaded with a NOP and `valid = 0`, an empty slot (a bubble).

11.6 ID/EX pipeline register (pipeline_register.v)

```
module pipeline_register (
    input             clk,
    input             rst,
    input             flush,
    // ---- from DECODE ----
    input      [2:0]  alu_ctrl_in,
    input             alu_src_sel_in, wb_sel_in, reg_we_in, mem_we_in,
    input             zero_we_in, carry_we_in, is_jmp_in, is_jz_in,
    input      [1:0]  rd_in, rs_in,
    input      [3:0]  imm_in, pc_in,
    input             valid_in,
    input      [11:0] instr_in,
    // ---- to EXECUTE ----
    output reg [2:0]  alu_ctrl,
    output reg        alu_src_sel, wb_sel, reg_we, mem_we,
    output reg        zero_we, carry_we, is_jmp, is_jz,
    output reg [1:0]  rd, rs,
    output reg [3:0]  imm, pc,
    output reg        valid,
    output reg [11:0] instr
);
    initial begin
```

```
        {alu_ctrl, alu_src_sel, wb_sel, reg_we, mem_we,
         zero_we, carry_we, is_jmp, is_jz} = 11'd0;
        {rd, rs, imm, pc} = 12'd0;
        valid = 1'b0;
        instr = 12'hC00;
    end

    always @(posedge clk) begin
        if (rst | flush) begin                      // bubble: EX changes nothing
            {alu_ctrl, alu_src_sel, wb_sel, reg_we, mem_we,
             zero_we, carry_we, is_jmp, is_jz} <= 11'd0;
            {rd, rs, imm, pc} <= 12'd0;
            valid <= 1'b0;
            instr <= 12'hC00;
        end else begin                              // pass the decoded instruction
on
            {alu_ctrl, alu_src_sel, wb_sel, reg_we, mem_we,
             zero_we, carry_we, is_jmp, is_jz}
                <= {alu_ctrl_in, alu_src_sel_in, wb_sel_in, reg_we_in, mem_we_in,
                    zero_we_in, carry_we_in, is_jmp_in, is_jz_in};
            {rd, rs, imm, pc} <= {rd_in, rs_in, imm_in, pc_in};
            valid <= valid_in;
            instr <= instr_in;
        end
    end
endmodule
```

The `_in` signals from the ID stage are copied to the outputs for the EX stage at every clock, grouped with concatenation braces. On reset or flush every control signal becomes 0, so the EX stage writes nothing.

11.7 Data memory (data_memory.v)

```
module data_memory (
    input        clk,
    input  [3:0] address,
    input  [3:0] write_data,
    input        mem_we,
    output [3:0] read_data
);
    reg [3:0] mem [0:15];
    integer m;

    initial for (m = 0; m < 16; m = m + 1) mem[m] = 4'b0000;

    always @(posedge clk)                // write on the clock edge (STORE)
        if (mem_we) mem[address] <= write_data;

    assign read_data = mem[address];     // read any time (LOAD)
endmodule
```

The data memory is separate from the instruction memory (Harvard organisation). Reading needs no clock; writing happens only on a clock edge with `mem_we = 1`.

11.8 Multiplexer (mux.v)

```
module mux2 (
    input  [3:0] d0,
    input  [3:0] d1,
    input        sel,
    output [3:0] y
);
    assign y = sel ? d1 : d0;
endmodule
```

The CPU uses this module three times: the operand MUX, the write-back MUX and the PC MUX.

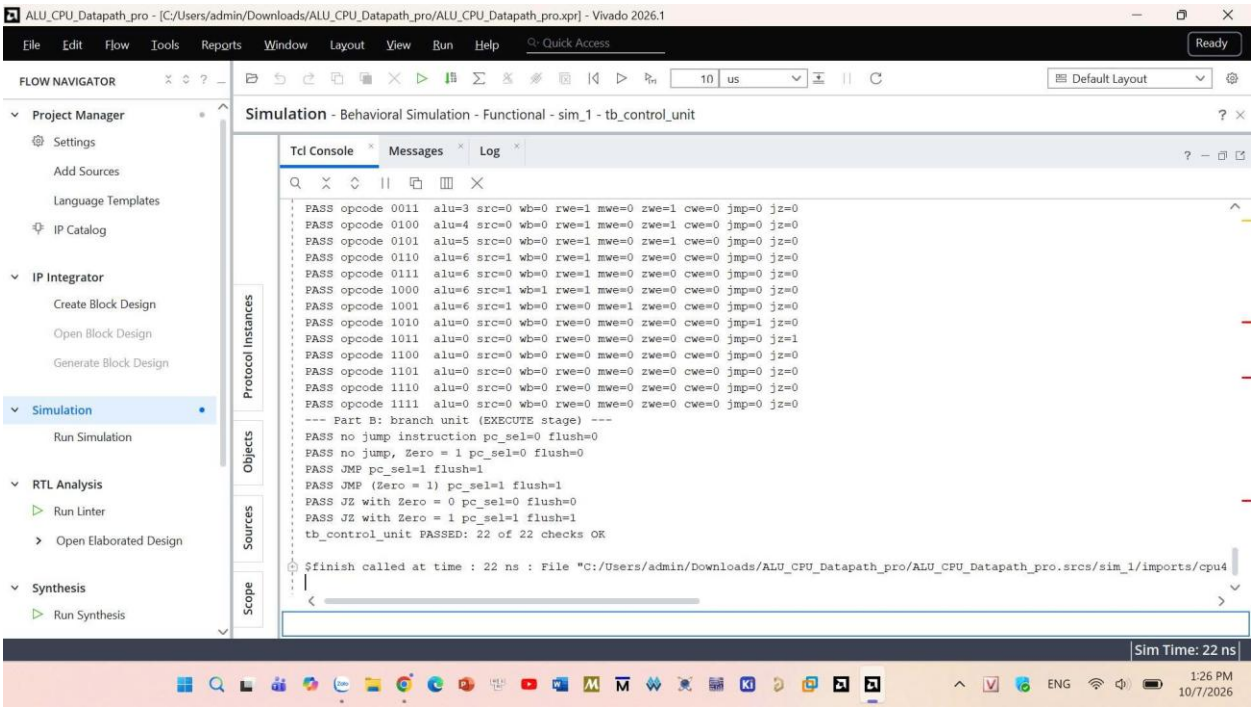
11.9 Flag register (flag_register.v)

```
module flag_register (
    input      clk,
    input      rst,
    input      zero_we,
    input      carry_we,
    input      zero_in,
    input      carry_in,
    output reg zero_flag,
    output reg carry_flag
);
    initial begin
        zero_flag  = 1'b0;
        carry_flag = 1'b0;
    end

    always @(posedge clk) begin
        if (rst) begin
            zero_flag  <= 1'b0;
            carry_flag <= 1'b0;
        end else begin
            zero_flag  <= zero_we  ? zero_in  : zero_flag;
            carry_flag <= carry_we ? carry_in : carry_flag;
        end
    end
endmodule
```

The two flags have separate enables because AND, OR, XOR and NOT change only Zero, while ADD and SUB change both Zero and Carry.

11.10 Control unit (control_unit.v)




```
// Control word (11 bits):
//  {alu_ctrl[2:0], alu_src_sel, wb_sel, reg_we, mem_we,
//   zero_we, carry_we, is_jmp, is_jz}
// Reserved opcodes 1100-1111 give the all-zero word = NOP.
module instruction_decoder (
    input  [3:0] opcode,
    output [2:0] alu_ctrl,
    output       alu_src_sel, wb_sel, reg_we, mem_we,
    output       zero_we, carry_we, is_jmp, is_jz
);
    localparam [2:0] A_ADD = 3'd0, A_SUB = 3'd1, A_AND = 3'd2, A_OR  = 3'd3,
                     A_XOR = 3'd4, A_NOT = 3'd5, A_PASS = 3'd6;

    reg [10:0] cw;
    assign {alu_ctrl, alu_src_sel, wb_sel, reg_we, mem_we,
            zero_we, carry_we, is_jmp, is_jz} = cw;

    //                    alu     src wb  rwe mwe zwe cwe jmp jz
    always @(*) begin
        case (opcode)
            4'b0000: cw = {A_ADD , 1'b0,1'b0,1'b1,1'b0,1'b1,1'b1,1'b0,1'b0}; // ADD
            4'b0001: cw = {A_SUB , 1'b0,1'b0,1'b1,1'b0,1'b1,1'b1,1'b0,1'b0}; // SUB
            4'b0010: cw = {A_AND , 1'b0,1'b0,1'b1,1'b0,1'b1,1'b0,1'b0,1'b0}; // AND
            4'b0011: cw = {A_OR  , 1'b0,1'b0,1'b1,1'b0,1'b1,1'b0,1'b0,1'b0}; // OR
            4'b0100: cw = {A_XOR , 1'b0,1'b0,1'b1,1'b0,1'b1,1'b0,1'b0,1'b0}; // XOR
            4'b0101: cw = {A_NOT , 1'b0,1'b0,1'b1,1'b0,1'b1,1'b0,1'b0,1'b0}; // NOT
            4'b0110: cw = {A_PASS, 1'b1,1'b0,1'b1,1'b0,1'b0,1'b0,1'b0,1'b0}; // LDI
            4'b0111: cw = {A_PASS, 1'b0,1'b0,1'b1,1'b0,1'b0,1'b0,1'b0,1'b0}; // MOV
            4'b1000: cw = {A_PASS, 1'b1,1'b1,1'b1,1'b0,1'b0,1'b0,1'b0,1'b0}; // LOAD
            4'b1001: cw = {A_PASS, 1'b1,1'b0,1'b0,1'b1,1'b0,1'b0,1'b0,1'b0}; // STORE
```

```
            4'b1010: cw = {A_ADD , 1'b0,1'b0,1'b0,1'b0,1'b0,1'b0,1'b1,1'b0}; // JMP
            4'b1011: cw = {A_ADD , 1'b0,1'b0,1'b0,1'b0,1'b0,1'b0,1'b0,1'b1}; // JZ
            default: cw = 11'b000_0000_0000;                                  //
reserved = NOP
        endcase
    end
endmodule

module branch_unit (
    input  is_jmp,
    input  is_jz,
    input  zero_flag,
    output pc_sel,
    output flush
);
    assign pc_sel = is_jmp | (is_jz & zero_flag);   // JMP always, JZ only if Zero =
1
    assign flush  = pc_sel;                          // jump taken -> flush
endmodule

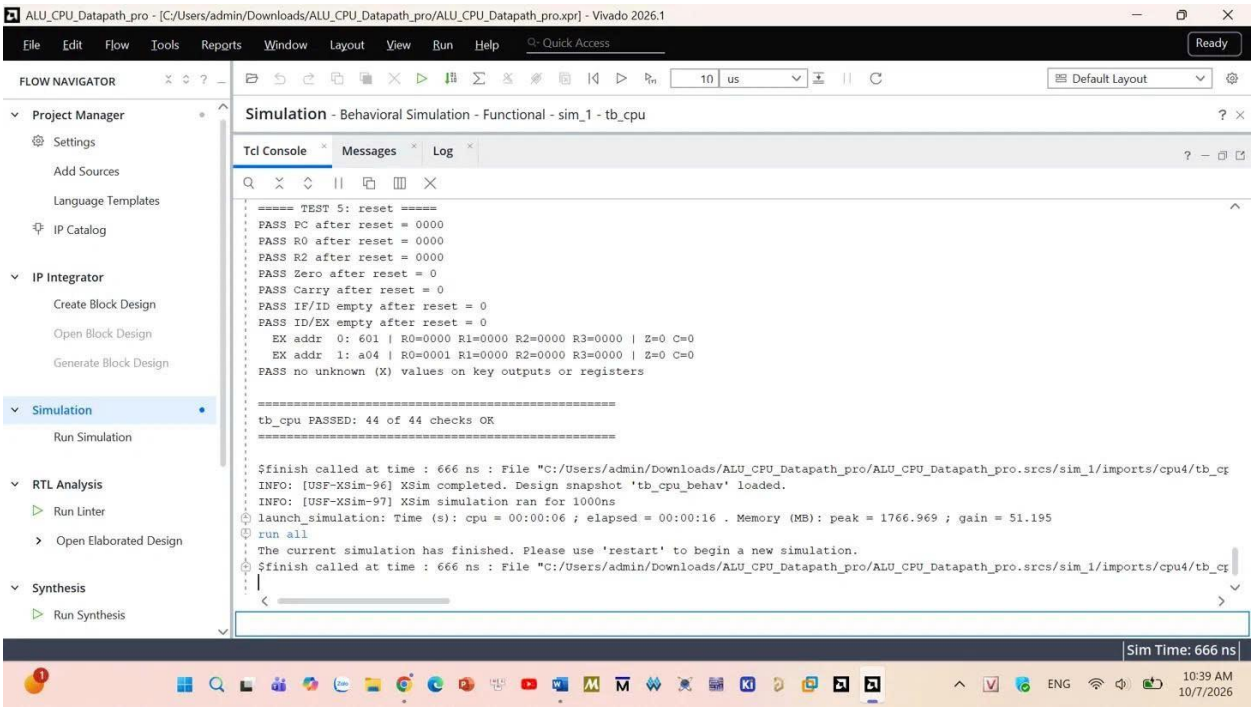
module control_unit (
    // ---- DECODE stage ----
    input  [3:0] opcode,
    output [2:0] alu_ctrl,
    output       alu_src_sel, wb_sel, reg_we, mem_we,
    output       zero_we, carry_we, is_jmp, is_jz,
    // ---- EXECUTE stage ----
    input        ex_is_jmp,
    input        ex_is_jz,
    input        zero_flag,
    output       pc_sel,
    output       flush
);
    instruction_decoder DECODER (
        .opcode(opcode), .alu_ctrl(alu_ctrl), .alu_src_sel(alu_src_sel),
        .wb_sel(wb_sel), .reg_we(reg_we), .mem_we(mem_we),
        .zero_we(zero_we), .carry_we(carry_we),
        .is_jmp(is_jmp), .is_jz(is_jz)
    );
    branch_unit BRANCH (
        .is_jmp(ex_is_jmp), .is_jz(ex_is_jz), .zero_flag(zero_flag),
        .pc_sel(pc_sel), .flush(flush)
    );
endmodule
```

- **instruction_decoder:** a `case` on the opcode returns one 11-bit control word, which is split into the individual signals. Named `localparam` constants keep the table readable, and the `default` branch makes opcodes 1100-1111 a NOP.
- **branch_unit:** one expression sets `pc_sel` for JMP, or for JZ when Zero = 1, and `flush` follows it.
- **control_unit:** joins the two. Its decoder ports connect to the ID stage and its branch ports to the EX stage.

11.11 Top level (cpu_top.v)

Part of cpu_top.v	Blocks used	What happens
STAGE 1: IF	mux2 (PC MUX), program_counter, instruction_memory, instruction_register	Fetch the instruction at PC into IF/ID; PC <- PC + 1 or jump address
STAGE 2: ID	control_unit (decoder), pipeline_register	Decode the opcode; save control + fields in ID/EX
STAGE 3: EX	register_file, mux2 (operand), alu, flag_register, data_memory, mux2 (write-back), control_unit (branch unit)	Read registers, compute, access memory, write back, update flags, decide jumps

Table 27. The three stages in cpu_top.v




```
module cpu_top #(
    parameter PROGRAM_FILE = "program.mem"
)(
    input         clk,
    input         rst,
    output [3:0]  pc_out,        // fetch address
    output [11:0] instr_out,     // instruction currently in EX
    output [3:0]  alu_out,       // ALU result
    output [3:0]  wb_data_out,   // value going into Rd
    output        zero_out,      // stored Zero flag
    output        carry_out      // stored Carry flag
```

```
);
    // ------------------------ signal declarations ------------------------
    wire [3:0]  pc, pc_plus_1, pc_next;               // IF
    wire [11:0] fetched;
    wire [11:0] id_instr;                              // IF/ID
    wire [3:0]  id_pc;
    wire        id_valid;
    wire [3:0]  id_opcode = id_instr[11:8];            // ID
    wire [1:0]  id_rd     = id_instr[7:6];
    wire [1:0]  id_rs     = id_instr[5:4];
    wire [3:0]  id_imm    = id_instr[3:0];
    wire [2:0]  id_alu_ctrl;
    wire        id_alu_src_sel, id_wb_sel, id_reg_we, id_mem_we;
    wire        id_zero_we, id_carry_we, id_is_jmp, id_is_jz;
    wire [2:0]  ex_alu_ctrl;                           // ID/EX
    wire        ex_alu_src_sel, ex_wb_sel, ex_reg_we, ex_mem_we;
    wire        ex_zero_we, ex_carry_we, ex_is_jmp, ex_is_jz;
    wire [1:0]  ex_rd, ex_rs;
    wire [3:0]  ex_imm, ex_pc;
    wire        ex_valid;
    wire [11:0] ex_instr;
    wire [3:0]  rd_value, rs_value, alu_b, alu_result, mem_data, wb_data;  // EX
    wire        alu_zero, alu_carry;
    wire        zero_flag, carry_flag;
    wire        pc_sel, flush;

    // =========================== STAGE 1 : IF ============================
    assign pc_plus_1 = pc + 4'd1;                       // wraps 15 -> 0

    mux2 PC_MUX (.d0(pc_plus_1), .d1(ex_imm), .sel(pc_sel), .y(pc_next));

    program_counter PC (.clk(clk), .rst(rst), .pc_en(1'b1),
                        .pc_next(pc_next), .pc_out(pc));

    instruction_memory #(.PROGRAM_FILE(PROGRAM_FILE)) IMEM (
        .address(pc), .instruction(fetched));

    instruction_register IR (
        .clk(clk), .rst(rst), .ir_load(1'b1), .flush(flush),
        .instruction_in(fetched), .pc_in(pc),
        .instruction_out(id_instr), .pc_out(id_pc), .valid_out(id_valid));

    // =========================== STAGE 2 : ID ============================
    control_unit CU (
        .opcode(id_opcode),
        .alu_ctrl(id_alu_ctrl), .alu_src_sel(id_alu_src_sel),
        .wb_sel(id_wb_sel), .reg_we(id_reg_we), .mem_we(id_mem_we),
        .zero_we(id_zero_we), .carry_we(id_carry_we),
        .is_jmp(id_is_jmp), .is_jz(id_is_jz),
        .ex_is_jmp(ex_is_jmp), .ex_is_jz(ex_is_jz), .zero_flag(zero_flag),
        .pc_sel(pc_sel), .flush(flush));

    pipeline_register ID_EX (
        .clk(clk), .rst(rst), .flush(flush),
        .alu_ctrl_in(id_alu_ctrl), .alu_src_sel_in(id_alu_src_sel),
```

```
        .wb_sel_in(id_wb_sel), .reg_we_in(id_reg_we), .mem_we_in(id_mem_we),
        .zero_we_in(id_zero_we), .carry_we_in(id_carry_we),
        .is_jmp_in(id_is_jmp), .is_jz_in(id_is_jz),
        .rd_in(id_rd), .rs_in(id_rs), .imm_in(id_imm),
        .pc_in(id_pc), .valid_in(id_valid), .instr_in(id_instr),
        .alu_ctrl(ex_alu_ctrl), .alu_src_sel(ex_alu_src_sel),
        .wb_sel(ex_wb_sel), .reg_we(ex_reg_we), .mem_we(ex_mem_we),
        .zero_we(ex_zero_we), .carry_we(ex_carry_we),
        .is_jmp(ex_is_jmp), .is_jz(ex_is_jz),
        .rd(ex_rd), .rs(ex_rs), .imm(ex_imm),
        .pc(ex_pc), .valid(ex_valid), .instr(ex_instr));

    // ========================== STAGE 3 : EX ==========================
    register_file RF (
        .clk(clk), .rst(rst),
        .ra(ex_rd), .rb(ex_rs),            // A = Rd, B = Rs
        .wa(ex_rd), .wd(wb_data), .we(ex_reg_we),
        .qa(rd_value), .qb(rs_value));

    mux2 OPERAND_MUX (.d0(rs_value), .d1(ex_imm), .sel(ex_alu_src_sel), .y(alu_b));

    alu ALU (.a(rd_value), .b(alu_b), .alu_ctrl(ex_alu_ctrl),
             .result(alu_result), .carry(alu_carry), .zero(alu_zero));

    flag_register FLAGS (
        .clk(clk), .rst(rst),
        .zero_we(ex_zero_we), .carry_we(ex_carry_we),
        .zero_in(alu_zero), .carry_in(alu_carry),
        .zero_flag(zero_flag), .carry_flag(carry_flag));

    data_memory DMEM (
        .clk(clk), .address(alu_result), .write_data(rd_value),
        .mem_we(ex_mem_we), .read_data(mem_data));

    mux2 WB_MUX (.d0(alu_result), .d1(mem_data), .sel(ex_wb_sel), .y(wb_data));

    // ------------------------------ debug outputs ------------------------------
    assign pc_out      = pc;
    assign instr_out   = ex_instr;
    assign alu_out     = alu_result;
    assign wb_data_out = wb_data;
    assign zero_out    = zero_flag;
    assign carry_out   = carry_flag;
endmodule
```

`cpu_top.v` only connects blocks. All wires are declared first, then the three stages follow in order. The jump address comes from `ex_imm`, the immediate of the instruction in EX, and `pc_sel` and `flush` go back to the PC MUX and to both pipeline registers.

12. Programs

12.1 Writing a program by hand

Programs are written as hexadecimal machine code in a `.mem` file, one 3-digit word per line from address 0. Look up the opcode (section 6.2), write Rd and Rs as 2-bit numbers (unused fields = 00), add the 4-bit immediate or address, then turn each group of 4 bits into one hex digit. Unused words are C00 (NOP). The four `.mem` files in this project are padded to 16 lines with C00.

12.2 Demonstration program (program.mem, plan Appendix A)

Addr	Hex	Instruction	Purpose
0	645	LDI R1, 0101	Load decimal 5 into R1
1	683	LDI R2, 0011	Load decimal 3 into R2
2	060	ADD R1, R2	R1 becomes 1000 (decimal 8)
3	6CA	LDI R3, 1010	Load a bit pattern into R3
4	470	XOR R1, R3	R1 becomes 0010 (decimal 2)
5	A05	JMP 0101	Loop at the last instruction for a stable demonstration

Table 28. Demonstration program

12.3 Test program: arithmetic and memory (prog_arith.mem)

Addr	Hex	Instruction	Effect
0	645	LDI R1, 5	R1 = 0101
1	683	LDI R2, 3	R2 = 0011
2	060	ADD R1, R2	R1 = 1000, Carry = 0, Zero = 0
3	940	STORE R1, 0	data_memory[0] = 1000
4	645	LDI R1, 5	R1 = 0101
5	160	SUB R1, R2	R1 = 0010, Zero = 0, Carry = 1 (no borrow)
6	941	STORE R1, 1	data_memory[1] =

Addr	Hex	Instruction	Effect
			0010
7	800	LOAD R0, 0	R0 = 1000 (reads what STORE wrote 4 instructions earlier)
8	1A0	SUB R2, R2	R2 = 0000, Zero = 1
9	A09	JMP 9	Stop

Table 29. Arithmetic and memory test program

12.4 Test program: logic operations (prog_logic.mem)

Addr	Hex	Instruction	Effect
0	64C	LDI R1, 1100	R1 = 1100
1	68A	LDI R2, 1010	R2 = 1010
2	7D0	MOV R3, R1	R3 = 1100 (uses R1 written just one instruction before)
3	2E0	AND R3, R2	R3 = 1000
4	7D0	MOV R3, R1	R3 = 1100
5	3E0	OR R3, R2	R3 = 1110
6	460	XOR R1, R2	R1 = 0110
7	790	MOV R2, R1	R2 = 0110
8	605	LDI R0, 0101	R0 = 0101
9	500	NOT R0	R0 = 1010
10	6CA	LDI R3, 1010	R3 = 1010
11	4F0	XOR R3, R3	R3 = 0000, Zero = 1
12	A0C	JMP 12	Stop

Table 30. Logic test program

12.5 Test program: PC and jumps (prog_jump.mem)

Addr	Hex	Instruction	Effect
0	601	LDI R0, 1	R0 = 1; PC 0 -> 1

Addr	Hex	Instruction	Effect
			(increment)
1	A04	JMP 4	PC becomes 0100; addresses 2 and 3 are flushed
2-3	60F	LDI R0, 15	must never run
4	647	LDI R1, 7	R1 = 0111
5	150	SUB R1, R1	R1 = 0000, Zero = 1
6	B09	JZ 9	taken: PC = 9; addresses 7 and 8 are flushed
7-8	68F	LDI R2, 15	must never run
9	6C2	LDI R3, 2	R3 = 0010
10	0F0	ADD R3, R3	R3 = 0100, Zero = 0
11	B0E	JZ 14	not taken: continue with 12
12	686	LDI R2, 6	R2 = 0110
13	A0D	JMP 13	Stop
14	68F	LDI R2, 15	must never run

Table 31. PC and jump test program

13. Verification

Testing is done at two levels, as the project plan asks: **module-level tests** that check each block on its own, and **CPU-level tests** that run programs from instruction memory. Every testbench is self-checking: it compares results with expected values, prints PASS or FAIL for each check and ends with a summary line.

Level	Testbench	What it checks	Checks
Module	tb_alu.v	All ALU operations, Zero, Carry / no-borrow	11
Module	tb_register_file.v	Reset, write and read of every register, both ports, we = 0	11

Level	Testbench	What it checks	Checks
Module	tb_control_unit.v	Decoder output for all 16 opcodes; branch unit in all 6 situations	22
CPU	tb_cpu.v	Demo program, arithmetic, memory, logic, MOV, LDI, Zero, PC increment, jumps and flushing, reset, no X values	44
Total			**88**

Table 32. Testbenches

13.1 Test plan coverage (project plan Section 8)

Plan test	Setup	Expected	Where tested	Result
ADD	R1 = 0101, R2 = 0011; ADD R1,R2	R1 = 1000, Carry 0, Zero 0	tb_alu, tb_cpu T2	PASS
SUB	R1 = 0101, R2 = 0011; SUB R1,R2	R1 = 0010, Zero 0 (Carry 1)	tb_alu, tb_cpu T2	PASS
AND	1100 AND 1010	1000	tb_alu, tb_cpu T3	PASS
OR	1100 OR 1010	1110	tb_alu, tb_cpu T3	PASS
XOR	1100 XOR 1010	0110	tb_alu, tb_cpu T3	PASS
NOT	NOT 0101	1010	tb_alu, tb_cpu T3	PASS
LDI	LDI R3, 1010	R3 = 1010	tb_cpu T1, T3	PASS
MOV	R1 = 0110; MOV R2,R1	R2 = 0110	tb_cpu T3	PASS
Zero flag	Result 0000	Zero = 1	tb_alu, tb_cpu T2, T3	PASS
PC increment	Reset, run in order	PC 0 -> 1	tb_cpu T4	PASS
Jump / branch	JMP / taken JZ to address 4	PC = 0100	tb_cpu T4	PASS
Reset	Assert reset	PC, registers, flags, pipeline at	tb_cpu T5	PASS

Plan test	Setup	Expected	Where tested	Result
		initial values		

Table 33. Every test in the project plan, with its result

13.2 Module-level testbenches

tb_alu applies 11 input vectors, each held for 10 ns, and compares `{result, carry, zero}` with the expected values through one reusable `run` task:

```
task run;
    input [8*12:1] name;
    input [3:0] ia, ib;
    input [2:0] op;
    input [3:0] e_res;
    input       e_cy, e_z;
    begin
        a = ia; b = ib; alu_ctrl = op; #10;
        tests = tests + 1;
        if ({result, carry, zero} === {e_res, e_cy, e_z}) // PASS line
        else errors = errors + 1;                         // FAIL line
    end
endtask

//  name           a      b      op    result C Z
run("ADD 5+3     ", 4'b0101, 4'b0011, 3'd0, 4'b1000, 1'b0, 1'b0);
run("ADD 15+1    ", 4'b1111, 4'b0001, 3'd0, 4'b0000, 1'b1, 1'b1);
run("SUB 5-3     ", 4'b0101, 4'b0011, 3'd1, 4'b0010, 1'b1, 1'b0);
run("SUB 3-5     ", 4'b0011, 4'b0101, 3'd1, 4'b1110, 1'b0, 1'b0);
run("SUB 7-7     ", 4'b0111, 4'b0111, 3'd1, 4'b0000, 1'b1, 1'b1);
// ... AND, OR, XOR, XOR to zero, NOT, PASS b (11 vectors in total)
```

tb_register_file has a clock and reset. A `put` task writes one register per clock edge and an `expect4` task compares a read value. The sequence is: check reset values on ports A and B (2 checks); write R0-R3 with 1, 5, A, F; read them back on both ports (6 checks); try a write with `we = 0` and check the register is unchanged (1 check); reset again and check two registers are 0 (2 checks).

tb_control_unit stores the decoder truth table (Table 17) in an array of 16 expected control words and loops over all 16 opcodes (16 checks). A `branch_case` task then checks `pc_sel` and `flush` in the six situations of Table 18 (6 checks):

```
reg [10:0] want [0:15];                 // expected control word per opcode
initial begin
    want[0] = 11'b000_0_0_1_0_1_1_0_0;  // ADD
    want[1] = 11'b001_0_0_1_0_1_1_0_0;  // SUB
    // ... one line per opcode, 1100-1111 = all zeros
end
for (k = 0; k < 16; k = k + 1) begin
    opcode = k[3:0]; #1;
    if (got === want[k]) /* PASS */ else /* FAIL */;
end
```

13.3 CPU-level testbench

- `load_prog` fills the instruction memory with NOP, clears the data memory and loads a `.mem` file; `hold_reset` holds reset for 2 clocks.
- `done(addr)` waits until the instruction fetched from that address is in EX and the next clock edge has written its result. Because of the pipeline, waiting a fixed number of cycles is not reliable after jumps, so every test waits for an instruction instead (with a 200-cycle guard against hangs).
- `chk4` / `chk1` compare a 4-bit or 1-bit value and print PASS or FAIL.
- Test 1 prints what each pipeline stage holds in every clock cycle; the other tests print one line per executed instruction. A cycle counter shows how many clocks a program took.
- `CPU.RF.regs[1]` is a hierarchical reference (inside CPU, inside RF, element 1 of the register array); it works in simulation only.
- The X check XOR-reduces all key bits: the result is X only if some bit is X.
- `$dumpfile("tb_cpu.vcd")` writes a waveform file that can be opened in GTKWave.

```
// wait until the instruction fetched from 'addr' has finished
task done;
    input [3:0] addr;
    begin
        guard = 0;
        @(negedge clk);
        while (!(CPU.ex_valid && CPU.ex_pc == addr) && guard < 200) begin
            @(negedge clk);
            guard = guard + 1;
        end
        @(posedge clk);
        #1;
    end
endtask
```

Test	Program	Checks	Covers
1	program.mem	6	LDI, ADD, XOR, JMP and the loop, pipeline trace
2	prog_arith.mem	11	ADD, SUB, Carry, Zero, STORE, LOAD
3	prog_logic.mem	8	AND, OR, XOR, MOV, NOT, LDI, Zero
4	prog_jump.mem	11	PC increment, JMP, JZ taken / not taken, flushing
5	reset	8	Reset values, empty pipeline, no X values
	Total	**44**	

Table 34. Checks in tb_cpu

14. Simulation Results

Every testbench was first run with Icarus Verilog and then with the Vivado Simulator (XSim) inside project `ALU_CPU_Datapath_pro`. Both gave the same results. The `$finish` times reported by Vivado were 110 ns (tb_alu), 67 ns (tb_register_file), 22 ns (tb_control_unit) and 666 ns (tb_cpu).

14.1 Module-level results

Testbench	Result	Notes
tb_alu	PASSED: 11 of 11	The 11 vectors appear in order in the waveform, 10 ns each
tb_register_file	PASSED: 11 of 11	R0-R3 take the values 1, 5, A, F; both reset events clear all registers
tb_control_unit	PASSED: 22 of 22	16 opcodes (1 ns each) then 6 branch cases

Table 35. Module-level results

[Insert Figure 11: Tcl Console of tb_alu, tb_register_file and tb_control_unit showing the PASSED lines (screenshots from your Vivado run)]

ALU waveform (110 ns). The values read directly from the waveform match the expected table:

Time (ns)	a	b	alu_ctrl	result	carry	zero	Test
0-10	5	3	0	8	0	0	ADD 5+3
10-20	F	1	0	0	1	1	ADD 15+1
20-30	5	3	1	2	1	0	SUB 5-3 (no borrow)
30-40	3	5	1	E	0	0	SUB 3-5 (borrow)
40-50	7	7	1	0	1	1	SUB 7-7
50-60	C	A	2	8	0	0	AND
60-70	C	A	3	E	0	0	OR
70-80	C	A	4	6	0	0	XOR
80-90	6	6	4	0	0	1	XOR to zero

Time (ns)	a	b	alu_ctrl	result	carry	zero	Test
90-100	5	0	5	A	0	0	NOT
100-110	0	9	6	9	0	0	PASS b

Table 36. ALU results read from the waveform

Register file waveform (67 ns). After the first reset (0-15 ns) all reads give 0. Four writes (`wa` = 0, 1, 2, 3 with `wd` = 1, 5, A, F) load the registers, and reading them back on both ports gives the same values. A write attempt with `we = 0` leaves R2 = A. The second reset (about 57 ns) clears all registers.

Control unit waveform (22 ns). `alu_ctrl` steps 0 to 5 for ADD to NOT and holds 6 (PASS) for LDI, MOV, LOAD and STORE; `alu_src_sel` is 1 for LDI, LOAD and STORE; `wb_sel` only for LOAD; `reg_we` for opcodes 0 to 8; `mem_we` only for STORE; `zero_we` for opcodes 0 to 5; `carry_we` only for ADD and SUB; `is_jmp` and `is_jz` for opcodes A and B; all signals are 0 for the reserved opcodes C to F.

14.2 CPU-level results

In the Test 1 trace each line shows one clock cycle: the address in IF, the instruction in ID, the instruction in EX and the register values during that cycle (a result appears in the next line, after the clock edge).

Cycle	IF (addr)	ID	EX	R1	R2	R3
1	0	bubble	bubble	0000	0000	0000
2	1	645 (addr 0)	bubble	0000	0000	0000
3	2	683 (addr 1)	645 (addr 0)	0000	0000	0000
4	3	060 (addr 2)	683 (addr 1)	0101	0000	0000
5	4	6CA (addr 3)	060 (addr 2)	0101	0011	0000
6	5	470 (addr 4)	6CA (addr 3)	1000	0011	0000
7	6	A05 (addr 5)	470 (addr 4)	1000	0011	1010
8	7	C00 (addr 6)	A05 (addr 5)	0010	0011	1010
9	5	bubble	bubble	0010	0011	1010
10	6	A05 (addr 5)	bubble	0010	0011	1010
11	7	C00 (addr 6)	A05 (addr 5)	0010	0011	1010

Table 37. Test 1 pipeline trace (demonstration program)

The table shows the pipeline filling (cycles 1-2), one instruction finishing per clock (cycles 3-7), the JMP in EX at cycle 8 sending the PC back to 5, and the two bubbles (cycles 9-10) before the JMP is in EX again at cycle 11.

Cycle counts measured in tb_cpu: **5 instructions finished after 7 clocks**, **9 instructions after 11 clocks** and **12 instructions after 14 clocks** (always N + 2 without jumps).

Tests 2 to 5 gave the expected register, memory, flag and PC values listed in section 12; for the jump program the final state is R0 = 0001, R2 = 0110 and the PC stops at 13. The final line of the CPU testbench was `tb_cpu PASSED: 44 of 44 checks OK` (simulation ended at 666 ns).

[Insert Figure 12: Vivado Tcl Console showing `tb_cpu PASSED: 44 of 44 checks OK`]

Overall: **88 of 88 checks passed** (11 ALU + 11 register file + 22 control unit + 44 CPU), with no unknown (X) values on the key outputs after reset.

14.3 Pipeline charts

Each row is one fetched instruction and each column a clock cycle; the coloured boxes show which stage the instruction is in.

Instruction (addr)	1	2	3	4	5	6	7	8	9	10
0	IF	ID	EX							
1		IF	ID	EX						
2			IF	ID	EX					
3				IF	ID	EX				
4					IF	ID	EX			
5 (JMP)						IF	ID	EX		
6							IF	ID	flush	
7								IF	flush	
5 again									IF	ID
6 again										IF

Table 38. Pipeline chart of the demonstration program (first 10 cycles). Instructions 0 to 4 overlap perfectly; JMP at address 5 flushes the two instructions fetched behind it.

For the jump program (`prog_jump.mem`) the JMP 4 (cycle 4) and the taken JZ 9 (cycle 9) flush two instructions each; the not-taken JZ 14 (address 11) costs nothing.

14.4 Waveform: demonstration program

[Insert Figure 13: Vivado waveform of tb_cpu, 0 to about 125 ns. Signals: clk, rst, pc_out, id_instr, instr_out, alu_out, wb_data_out, zero_out, carry_out, R1-R3, pc_sel, flush]

What the captured waveform shows:

- **Cycles 1-2:** the pipeline is filling; EX holds C00 (empty).
- **Cycle 3:** LDI R1 (645) is in EX; `alu_out` = 0101 and R1 becomes 0101 at the end of the cycle.
- **Cycles 3-7:** one instruction finishes every clock; `alu_out` / `wb_data_out` take the values 0101, 0011, 1000 (ADD), 1010 and 0010 (XOR), while the lines in `instr_out` run 645, 683, 060, 6CA, 470.
- **Cycle 8:** JMP (A05) is in EX; `pc_sel` = `flush` = 1, the PC returns to 5 and the next two slots are empty (C00).
- **From cycle 8:** the loop JMP 5 repeats every 3 cycles; `zero_out` and `carry_out` stay 0 because no ADD or SUB result is zero.

14.5 Waveform: jumps

[Insert Figure 14: Vivado waveform of tb_cpu during Test 4 (about 430 ns to 600 ns). Add zero_out, pc_sel and flush]

- **PC increment:** the PC goes 0, 1, 2 ... one step every clock.
- **JMP 4:** `pc_sel` = `flush` = 1; the PC becomes 4 and the ID / EX slots become empty.
- **JZ 9 taken:** Zero = 1 after SUB R1,R1, so the PC jumps to 9 and two instructions are flushed.
- **JZ 14 not taken:** Zero = 0 after ADD R3,R3, so `pc_sel` stays 0 and the pipeline continues without loss.
- **R2 = 0110:** only `LDI R2, 6` ever wrote R2; the three `LDI R2, 15` instructions were all removed.

14.6 Performance

Program	Instructions run	Old 2-cycle design	3-stage pipeline	Speed-up
Demonstration (to the XOR)	5	10 clocks	7 clocks	1.4x
Arithmetic and memory	9	18 clocks	11 clocks	1.6x
Logic operations	12	24 clocks	14 clocks	1.7x
Long program without jumps	N	2N clocks	N + 2 clocks	close to 2x

Table 39. Clock cycles needed (pipeline values measured in tb_cpu; the old design is a reference that needs 2 clocks per instruction)

14.7 Acceptance criteria (project plan Section 8.1)

Criterion	Status	Evidence

Criterion	Status	Evidence
All ALU operations give the expected 4-bit results	Met	tb_alu 11/11
Register reads and writes use the correct registers	Met	tb_register_file 11/11
Decoder gives the specified control signals for every opcode	Met	tb_control_unit 22/22
The CPU runs a test program in the intended order	Met	tb_cpu Tests 1-3, traces, pipeline charts
Jump and conditional-branch tests pass	Met	tb_cpu Test 4, jump waveform
No unwanted X values on key outputs after reset	Met	tb_cpu Test 5
Simulation evidence and test results in the report	Met	Chapters 13-14

Table 40. Acceptance criteria

15. Workflow in Vivado: Steps Carried Out, Problems and Fixes

This chapter records what was actually done in Vivado 2026.1, in order, so the work can be repeated and explained.

15.1 Overview of the steps

Step	Action	Result
1	Write and pre-check the Verilog code with Icarus Verilog	88 / 88 checks passed before opening Vivado
2	Create the Vivado project	`ALU_CPU_Datapath_pro`, Artix-7 xc7a12ticsg325-1L, language Verilog
3	Add the 11 design files and set `cpu_top` as top	Hierarchy shows cpu_top with PC, IMEM, IR, CU, ID_EX, RF, ALU, FLAGS, DMEM and the three MUXes
4	Add 4 testbenches and 4 `.mem` files as simulation sources	Simulation Sources: sim_1 holds 9 items
5	Run each testbench in the	tb_cpu 44/44; tb_alu, tb_register_file and

Step	Action	Result
	Vivado Simulator	tb_control_unit behave as expected
6	Build waveforms and save the waveform configuration	Figures for sections 14.4 and 14.5
7	Open the elaborated design and view the schematic	13 cells, 28 I/O ports, 116 nets
8	Fix the issues found (section 15.8)	Clean runs

Table 41. Workflow summary

15.2 Create the project

1. Start Vivado and choose **Create Project**, type **RTL Project**.
2. Name it `ALU_CPU_Datapath_pro` and place it in `C:/Users/admin/Downloads/ALU_CPU_Datapath_pro`.
3. Select the part **xc7a12ticsg325-1L** (Artix-7). No board is needed for simulation.
4. In the Project Summary the target language is Verilog and the top module is "Not defined" until step 3.

15.3 Add the design sources

1. In the Flow Navigator choose **Add Sources**, then *Add or create design sources*.
2. **Add Files** and select the 11 design files: `alu.v, control_unit.v, cpu_top.v, data_memory.v, flag_register.v, instruction_memory.v, instruction_register.v, mux.v, pipeline_register.v, program_counter.v, register_file.v`. No testbench or `.mem` file belongs in this list.
3. Tick **Copy sources into project**, so the files are stored inside the project (`ALU_CPU_Datapath_pro.srcs/sources_1/imports/cpu4/`) and the project can be moved or submitted as one folder.
4. Click **Finish**. Right-click `cpu_top` in the Sources window and choose **Set as Top**. When it is already top, the menu item is greyed out.

15.4 Add the simulation sources

1. **Add Sources**, then *Add or create simulation sources*.
2. Change the file filter to **All Files** so the `.mem` files are visible.
3. Select `tb_alu.v`, `tb_register_file.v`, `tb_control_unit.v`, `tb_cpu.v`, `program.mem`, `prog_arith.mem`, `prog_logic.mem` and `prog_jump.mem`; tick **Copy sources into project**; click **Finish**.
4. `sim_1` now lists the four testbenches and a Memory File group with the four `.mem` files.

15.5 Run the simulations

For each testbench:

1. Make sure no older simulation is open (**File -> Close Simulation**).
2. Right-click the testbench under Simulation Sources and choose **Set as Top**.
3. **Run Simulation -> Run Behavioral Simulation**. The title bar must show `sim_1 - <testbench name>`.
4. If the run does not end by itself, type `run all` in the Tcl Console.
5. Read the PASS lines and the final summary.

Testbench	Vivado finish time	Final summary
tb_cpu	666 ns	`tb_cpu PASSED: 44 of 44 checks OK`
tb_alu	110 ns	11 vectors, results match Table 36
tb_register_file	67 ns	11 checks, results match section 14.1
tb_control_unit	22 ns	22 checks, results match Tables 17 and 18

Table 42. Results of the simulation runs in Vivado

15.6 Build the waveform

The Objects window also lists internal testbench variables (`errors`, `checks`, `cycle`, `k`, `guard`, `trace_all`) that are not needed in the report. A clean waveform for `tb_cpu` is made like this: select all rows in the Name column and press Delete, then type in the Tcl Console:

```
add_wave /tb_cpu/clk /tb_cpu/rst
add_wave -radix hex /tb_cpu/CPU/id_instr /tb_cpu/CPU/ex_instr
add_wave -radix bin /tb_cpu/pc_out /tb_cpu/alu_out /tb_cpu/wb_data_out
/tb_cpu/zero_out /tb_cpu/carry_out
add_wave -radix bin /tb_cpu/CPU/RF/r1 /tb_cpu/CPU/RF/r2 /tb_cpu/CPU/RF/r3
add_wave /tb_cpu/CPU/pc_sel /tb_cpu/CPU/flush /tb_cpu/CPU/ex_reg_we
/tb_cpu/CPU/ex_alu_src_sel
```

Then click Restart, run (`run 200 ns` for Test 1, `run all` for everything), press Zoom Fit and save the waveform configuration (floppy-disk icon). The demonstration program (Test 1) occupies about 0-125 ns; the jump program (Test 4) about 430-600 ns. The waveform of `tb_cpu` gave the expected values: `pc_out` runs 0000 to 0111 and returns to 0101; `instr_out` runs 645, 683, 060, 6CA, 470, A05; `alu_out` gives 0101, 0011, 1000, 1010, 0010; `pc_sel` and `flush` rise when the JMP is in EX.

15.7 Elaborated design and schematic

1. In the Flow Navigator choose **RTL Analysis -> Open Elaborated Design**.
2. Open the **Schematic** of `cpu_top`. The elaborated design shows 13 cells, 28 I/O ports and 116 nets.
3. Capture the schematic for Figure 5, and capture the hierarchy tree in Sources for section 10.3.

15.8 Problems met and how they were solved

Problem	Cause	Fix
Error "boost::filesystem::remove: the process cannot access the file ... simulate.log" when starting a new simulation	The previous simulation was still open, so Vivado could not delete its log file	Close the simulation first (File -> Close Simulation, or `close_sim -force` in the Tcl Console). If it persists, close Explorer windows that show the `.sim` folder and end `xsim` / `xsimk` tasks in Task Manager
After setting a new top module the old testbench still ran	The running simulation keeps the old snapshot until it is closed	Close the simulation, set the new top, run Behavioral Simulation again; check the title bar
The waveform shows the same signals twice and many unneeded ones	Signals were added more than once; internal testbench variables are listed in Objects	Select all rows, Delete, then add only the needed signals with `add_wave` (section 15.6)
After `restart` all values are X and the time is 0	Restart only rewinds the simulation	Type `run all` (or `run 200 ns`)
Critical warning `[Synth 8-4445] could not open $readmem data file 'program.mem'` when opening the elaborated design	`program.mem` was only added to Simulation Sources, so the design cannot see it. The instruction memory would contain only C00 (NOP)	Add `program.mem` to Design Sources (Add Sources -> design sources -> All Files -> `program.mem` -> copy into project), set its file type to Memory File, then reload the elaborated design

Table 43. Problems and fixes

15.9 Optional: synthesis and FPGA demonstration

- **Before synthesis:** `program.mem` must be in Design Sources (previous table), otherwise the instruction memory is all NOP and most of the logic is optimised away, so the resource numbers would be wrong.
- **Clock constraint:** add a constraints file with `create_clock -period 10.000 -name clk [get_ports clk]`.
- **Run Synthesis** and read the Utilization report (LUT, FF) and the Timing Summary (WNS); `Run Implementation` and `Report Power` are available for the power figure.
- **FPGA demo:** divide the board clock down to about 1-2 Hz or use a debounced push-button as a single-step clock, connect `rst` to a push-button, show `pc_out`, `wb_data_out`, `zero_out` and `carry_out` on LEDs, and write an `.xdc` file that maps the ports to the board pins.

Item	Value

Item	Value
LUT	(fill in after synthesis)
FF	(fill in after synthesis)
WNS at 100 MHz	(fill in after synthesis)

Table 44. Synthesis results (optional)

Testbenches and hierarchical references (such as `CPU.RF.regs[1]`) are for simulation only; they are not part of the synthesised hardware.

15.10 Evidence to include in the final report

Evidence	Where it goes
Tcl Console lines with the four PASSED summaries (11, 11, 22, 44)	Section 14.1 and 14.2
Waveform of the demonstration program (0-125 ns)	Section 14.4
Waveform of the jump program (about 430-600 ns) with zero_out, pc_sel, flush	Section 14.5
Waveforms of tb_alu, tb_register_file and tb_control_unit	Section 14.1 (optional)
Schematic of cpu_top and the Sources hierarchy	Chapter 8 and section 10.3
Utilization and timing reports	Section 15.9 (optional)

Table 45. Evidence checklist

16. Limitations, Extensions and Conclusion

16.1 Limitations and possible extensions

Limitation	Cause	Possible extension
A taken jump costs 2 cycles	Jump decided in EX	Decide JMP in ID (1-cycle penalty) or add branch prediction
16 instructions per program	4-bit PC and jump field	Wider PC with wider jump field or relative jumps
16 data words	4-bit address field	Register-indirect addressing: LOAD Rd, [Rs]
Only one conditional jump	Plan baseline	Use reserved opcodes 1100-

Limitation	Cause	Possible extension
		1111 for JC, JNZ, shifts or HALT
No signed overflow / negative flags	Not in plan	Add V and N flags
No subroutines	No stack or link register	CALL / RET with a return-address register

Table 46. Limitations and possible extensions

16.2 Conclusion

This project designed, implemented and verified a 4-bit RISC-style CPU in Verilog HDL according to the project plan. The 12-bit instruction format and the 12-instruction ISA were fixed first. The datapath was built from a 4-bit ALU, a two-port register file, a program counter, a 16 x 12 instruction memory, a 16 x 4 data memory and three multiplexers, all controlled by a hardwired control unit, and organised as a 3-stage pipeline with an IF/ID register (the IR) and an ID/EX register.

Every instruction is really fetched, decoded, executed and written back, while the overlap of the stages lets one instruction finish every clock cycle. Data hazards are avoided by reading registers in EX, and control hazards are handled by flushing two instructions after a taken jump.

Three module-level testbenches and one CPU-level testbench ran 88 checks, all passing in both Icarus Verilog and the Vivado Simulator, covering every test and acceptance criterion in the plan. The waveforms and pipeline charts confirm the overlap of the stages and the correct flushing after jumps. The Vivado workflow also showed the practical points that matter when moving a design into a tool: close a simulation before starting another, keep waveforms clean, and add `$readmemh` files to the design sources.

Appendix A: Quick Reference Card

Mnemonic	Opcode	Format (binary)	Example	Hex
ADD Rd, Rs	0000	0000 dd ss 0000	ADD R1, R2	060
SUB Rd, Rs	0001	0001 dd ss 0000	SUB R1, R2	160
AND Rd, Rs	0010	0010 dd ss 0000	AND R3, R2	2E0
OR Rd, Rs	0011	0011 dd ss 0000	OR R3, R2	3E0
XOR Rd, Rs	0100	0100 dd ss 0000	XOR R1, R3	470
NOT Rd	0101	0101 dd 00 0000	NOT R0	500
LDI Rd, imm	0110	0110 dd 00 iiii	LDI R1, 5	645
MOV Rd, Rs	0111	0111 dd ss 0000	MOV R2, R1	790

Mnemonic	Opcode	Format (binary)	Example	Hex
LOAD Rd, a	1000	1000 dd 00 aaaa	LOAD R0, 0	800
STORE Rd, a	1001	1001 dd 00 aaaa	STORE R1, 1	941
JMP a	1010	1010 00 00 aaaa	JMP 5	A05
JZ a	1011	1011 00 00 aaaa	JZ 9	B09
(reserved)	1100-1111	-	NOP	C00

Table 47. Instruction encoding quick reference (dd = Rd, ss = Rs)

Register	Code
R0	00
R1	01
R2	10
R3	11

Pipeline fact	Value
Stages	IF, ID, EX
Latency of one instruction	3 clocks
Throughput (no jumps)	1 instruction per clock
N instructions, no jumps	N + 2 clocks
Penalty of a taken jump	2 clocks (2 instructions flushed)
Penalty of a not-taken JZ	0 clocks

Appendix B: How to Repeat the Simulations

Icarus Verilog (command line) in the folder that holds the 19 files:

```
iverilog -g2005 -o sim_tb_cpu tb_cpu.v alu.v register_file.v program_counter.v \
    instruction_memory.v instruction_register.v pipeline_register.v control_unit.v \
    data_memory.v mux.v flag_register.v cpu_top.v
vvp sim_tb_cpu
```

The script `run_all.sh` does this for all four testbenches and counts the PASS and FAIL lines (expected: 88 PASS, 0 FAIL).

Vivado: follow sections 15.2 to 15.5. Add `program.mem` to Design Sources if you will open the elaborated design or run synthesis.